

SECURE GUI-BASED MULTI-CLIENT TCP CHAT APPLICATION

USING PYTHON SOCKET PROGRAMMING

Submitted By

Name: Kunal Prajapati

Enrollment No.: UU2409000396

Uttaranchal University, Dehradun, Uttarakhand (248007)

GitHub Repository:

<https://github.com/kunalll0/ISEA-Phase3-TezpurUniversity-SyncChat>

Submitted To

ISEA Summer Internship 2026

ISEA under MeitY

(In affiliation with Tezpur University, Assam)

Course Assignment

Assignment 8

**SECURE GUI-BASED MULTI-CLIENT TCP CHAT APPLICATION
WITH PERFORMANCE OPTIMIZATION, SCALABILITY AND
RELIABILITY USING PYTHON SOCKET PROGRAMMING**

TABLE OF CONTENTS

Contents

1. OBJECTIVE	5
2. INTRODUCTION	7
3. PROBLEM STATEMENT	9
4. SYSTEM REQUIREMENTS.....	11
4.1 Functional Requirements	11
4.2 Non-Functional Requirements	12
5. SOFTWARE AND HARDWARE REQUIREMENTS	14
5.1 Software Requirements	14
5.2 Hardware Requirements	14
6. PROJECT ARCHITECTURE	15
6.1 Client Module.....	16
6.2 Server Module.....	17
6.3 Database and Configuration Module	17
7. DESIGN METHODOLOGY	19
8. FEATURES IMPLEMENTED	21
8.1 User Registration.....	21
8.2 Secure Login System.....	22
8.3 Duplicate Login Prevention	23
8.4 Login Attempt Restriction	24
8.5 Password Hashing	24
8.6 Online User Management.....	25
8.7 Broadcast Messaging	25
8.8 Private Messaging	26
8.9 Chat Commands.....	26
8.10 Message Length Validation	29
8.11 Connection Management	29
8.12 Reliability Enhancement	30
8.13 Configuration Management.....	30

8.14 Scalability Enhancement	30
8.15 Performance Evaluation	31
9. GUI DESIGN	32
9.1 Login Window	32
9.2 Signup Window	33
9.3 Chat Window.....	34
9.4 GUI Validation Messages.....	35
9.5 GUI Enhancements.....	36
10. SECURITY MECHANISMS	37
10.1 User Authentication	37
10.2 Password Hashing	37
10.3 Duplicate Username Prevention	38
10.4 Duplicate Login Prevention	38
10.5 Login Attempt Restriction	39
10.6 Session Timeout.....	39
10.7 Message Length Validation	40
10.8 Secure Logging	40
10.9 Chat History.....	41
10.10 Connection Reliability	42
10.11 Configuration Security	42
10.12 Security Monitoring During Performance Evaluation	43
11. EXPERIMENTAL RESULTS AND TESTING	44
11.1 User Registration Testing.....	44
11.2 User Authentication Testing.....	45
11.3 Duplicate Login Testing	46
11.4 Chat Communication Testing	46
11.5 Online User Management Testing.....	47
11.6 Server Command Testing	48
11.7 Message Length Validation Testing.....	50
11.8 Session Timeout Testing.....	51
11.9 Configuration Management Testing.....	51
11.10 Connection Reliability Testing	52

11.11 Scalability Testing	53
12. WIRESHARK PACKET ANALYSIS	54
12.1 TCP Three-Way Handshake	54
12.2 Broadcast Message Packet	55
12.3 Private Message Packet	55
12.4 User Disconnection Packet	56
12.5 TCP Connection Termination	56
12.6 Logout Packet	57
13. LOG FILE ANALYSIS	58
13.1 User Database (users.csv)	58
13.2 Chat History (chat_history.csv)	59
13.3 Security Log (security_log.txt)	60
13.4 Server Runtime Statistics	61
13.5 Configuration File (config.json)	62
13.6 Performance Results File (performance_results.csv)	63
13.7 Benchmark Graph Analysis	63
14. CHALLENGES FACED	65
15. LEARNING OUTCOMES	67
16. CONCLUSION	69
17. REFERENCES	70

1. OBJECTIVE

The primary objective of this assignment is to design and develop a **Secure GUI-Based Multi-Client Chat Application** using Python Socket Programming. Unlike the previous assignment that focused mainly on socket communication, this project integrates networking with authentication, graphical user interface development, secure password management, session handling, message logging, and packet analysis.

The application enables multiple authenticated users to communicate simultaneously over a TCP network while maintaining confidentiality, integrity, and controlled access. Several cybersecurity mechanisms have been incorporated to simulate the behaviour of a secure real-world communication platform.

The specific objectives of this assignment are:

- Develop a GUI-based chat application using Tkinter.
- Implement secure client-server communication using TCP sockets.
- Provide account registration and authenticated login.
- Securely store passwords using SHA-256 hashing.
- Prevent brute-force attacks through login attempt limiting.
- Prevent duplicate simultaneous logins.
- Implement automatic inactivity logout.
- Support broadcast and private messaging.
- Maintain complete chat history.
- Generate separate security logs for auditing.
- Display active online users dynamically.
- Verify network communication using Wireshark packet analysis.

The project successfully combines concepts from Python programming, computer networks, cybersecurity, GUI development, and secure software engineering into a single integrated application.

This assignment focuses on improving the overall software quality of the Secure SyncChat application through optimization, scalability enhancement, and reliability improvement.

The existing client-server architecture was retained while implementing better connection management, enhanced exception handling, centralized configuration management, and resource optimization.

Performance benchmarking was conducted using 5, 8, and 10 concurrent clients in a Mininet environment. System performance was evaluated by measuring message delay, throughput,

CPU utilization, and memory consumption, with results recorded in `performance_results.csv` and visualized using graphical analysis.

These enhancements make the application more scalable, reliable, maintainable, and suitable for handling larger numbers of simultaneous users.

2. INTRODUCTION

Communication systems are among the most important applications of computer networks. Modern messaging platforms such as WhatsApp, Microsoft Teams, Discord, and Slack rely on secure client-server communication to exchange information among multiple users. These applications authenticate users, maintain activity logs, manage user sessions, support real-time interaction, and are designed to remain reliable and responsive even when multiple users communicate simultaneously.

This project aims to replicate the core functionality of such communication platforms in a simplified academic environment using Python. The focus is not only on implementing secure client-server communication but also on developing a reliable, scalable, and user-friendly messaging system capable of supporting multiple concurrent users.

The developed application, named **SyncChat**, is a secure GUI-based multi-client messaging platform built using Python's Socket Programming library and the Tkinter graphical user interface framework. Unlike traditional command-line chat applications, SyncChat provides a complete desktop interface that enables users to register, authenticate themselves, exchange broadcast and private messages, monitor online users, and retrieve useful server information through an intuitive graphical environment.

Security has been incorporated as a fundamental aspect of the application. User passwords are never stored in plain text; instead, they are securely hashed using the SHA-256 cryptographic algorithm before being written to the local user database (users.csv). The application also includes password strength validation, duplicate account detection, login attempt restriction, duplicate session prevention, automatic inactivity logout, and comprehensive security logging to protect user accounts and improve accountability.

To improve software quality and support larger workloads, the application incorporates several reliability and scalability enhancements. A centralized configuration file is used to manage network parameters and application settings, simplifying deployment and maintenance. Improved connection management and exception handling ensure that unexpected client disconnections are handled gracefully without affecting other active users. The server is capable of supporting multiple concurrent client connections efficiently through multi-threading while maintaining stable communication and proper resource management.

Throughout execution, significant events such as account creation, successful and failed login attempts, logout operations, and session expiration are recorded in security_log.txt, while all chat communications are preserved in chat_history.csv. In addition, system performance is evaluated under different client loads by measuring message delivery delay, throughput, CPU utilization, and memory consumption. The collected performance data is stored in performance_results.csv and analyzed using graphical visualizations to evaluate the application's scalability and resource efficiency.

To validate the implementation, Wireshark was used to inspect the TCP packets exchanged between the clients and the server. Packet analysis confirmed successful TCP connection establishment through the three-way handshake, reliable message transmission, private

communication, graceful connection termination, and correct protocol behavior during normal application execution.

Overall, SyncChat demonstrates how computer networking, socket programming, graphical user interface development, cybersecurity practices, configuration management, performance optimization, and scalability evaluation can be integrated into a single client-server application. The project provides practical insight into designing secure, reliable, and scalable communication systems while following modern software engineering principles.

3. PROBLEM STATEMENT

Traditional socket programming examples generally focus on establishing communication between two endpoints. While such implementations demonstrate the fundamentals of network programming, they rarely include the security mechanisms, graphical interfaces, user management, reliability features, and performance optimization techniques required in modern communication systems.

A basic TCP chat application suffers from several limitations:

- Anyone can join the server without proper authentication.
- Passwords are either absent or stored insecurely.
- Multiple users may impersonate the same identity.
- Brute-force login attempts remain unrestricted.
- User activities cannot be monitored or audited effectively.
- No session management or inactivity handling exists.
- Users cannot easily identify active participants.
- Chat history is often lost after application termination.
- No graphical interface is available for user-friendly interaction.
- Network parameters are frequently hardcoded, making deployment and maintenance difficult.
- Unexpected client disconnections may interrupt communication or affect server stability.
- The application is rarely evaluated for scalability when multiple clients communicate simultaneously.
- Performance metrics such as message delay, throughput, CPU utilization, and memory consumption are generally not analyzed, making it difficult to assess system efficiency under increasing workloads.

Therefore, there is a need to design a secure, reliable, and scalable communication platform that combines networking functionality with modern software engineering and cybersecurity practices. Such a system should provide secure user authentication, efficient connection management, centralized configuration, persistent logging, graceful error handling, and the ability to maintain stable performance while supporting multiple concurrent users.

The proposed **SyncChat** application addresses these challenges by implementing secure authentication, SHA-256 password hashing, login validation, session management, detailed logging mechanisms, graphical user interfaces, centralized configuration management, improved connection handling, performance monitoring, and real-time multi-client communication. The application is further evaluated through scalability testing and network

packet analysis to ensure reliable operation and efficient resource utilization under different client loads.

.

4. SYSTEM REQUIREMENTS

The Secure GUI-Based Multi-Client TCP Chat Application was developed using Python and executed in a Linux environment. The project utilizes socket programming, multi-threading, graphical user interface development, cryptographic hashing, configuration management, and packet analysis tools to demonstrate secure, reliable, and scalable client-server communication.

The application was designed to operate efficiently on standard desktop systems without requiring high-end hardware or commercial software. In addition to providing secure communication, the system is capable of handling multiple concurrent client connections, maintaining reliable operation during unexpected network events, and evaluating performance under varying workloads through benchmarking and resource monitoring.

4.1 Functional Requirements

The application must satisfy the following functional requirements:

- User registration through a graphical interface.
- Secure login using registered credentials.
- Password hashing before storage.
- Duplicate username prevention.
- Duplicate login prevention.
- Broadcast messaging among all connected users.
- Private messaging between selected users.
- Dynamic display of online users.
- Automatic session timeout after inactivity.
- Logout functionality.
- Chat history maintenance.
- Security event logging.
- Server information retrieval.
- Help command support.
- Online user listing.
- Message length validation.
- Packet verification using Wireshark.
- Centralized configuration management using a configuration file.

- Graceful handling of client connections and disconnections.
 - Performance benchmarking under different client loads.
 - Generation of performance reports and graphical analysis.
-

4.2 Non-Functional Requirements

The application also satisfies several non-functional requirements.

Security

- Passwords are stored using SHA-256 hashing.
- Authentication is mandatory before accessing the chat room.
- Duplicate login attempts are blocked.
- Multiple failed login attempts trigger temporary account lock.
- Security events are recorded in a dedicated log file.

Reliability

- Multi-threaded server supports multiple clients simultaneously.
- Server remains operational even if one or more clients disconnect unexpectedly.
- Graceful connection handling prevents interruption of active user sessions.
- Chat history and security logs are preserved after application termination.
- Robust exception handling improves application stability during runtime.

Performance

- Messages are delivered with minimal delay.
- Online user updates occur dynamically.
- Session monitoring runs in the background without affecting communication.
- The application efficiently supports multiple concurrent clients with stable performance.
- CPU utilization, memory consumption, message throughput, and average message delay are monitored during benchmark testing.
- Performance results are recorded for further analysis and visualization.

Scalability

- The server is capable of supporting increasing numbers of concurrent client connections.

- The application has been evaluated under multiple client workloads to assess scalability and resource utilization.
- Communication performance remains stable as the number of connected clients increases.

Maintainability

- Network parameters and application settings are managed through a centralized configuration file.
- The modular design simplifies future feature additions and maintenance.
- Configuration changes can be applied without modifying the application source code.

Usability

- User-friendly graphical interface.
 - Simple navigation.
 - Clear validation messages.
 - Easy-to-understand controls.
 - Consistent interface design with real-time status updates.
-

5. SOFTWARE AND HARDWARE REQUIREMENTS

5.1 Software Requirements

Software	Purpose
Ubuntu (WSL)	Development Environment
Python 3	Programming Language
Tkinter	GUI Development
Socket Module	TCP Communication
Threading Module	Multi-client Handling
hashlib	Password Hashing
CSV Module	Data Storage
Wireshark	Packet Analysis
Mininet	Network Emulation
Visual Studio Code	Code Development

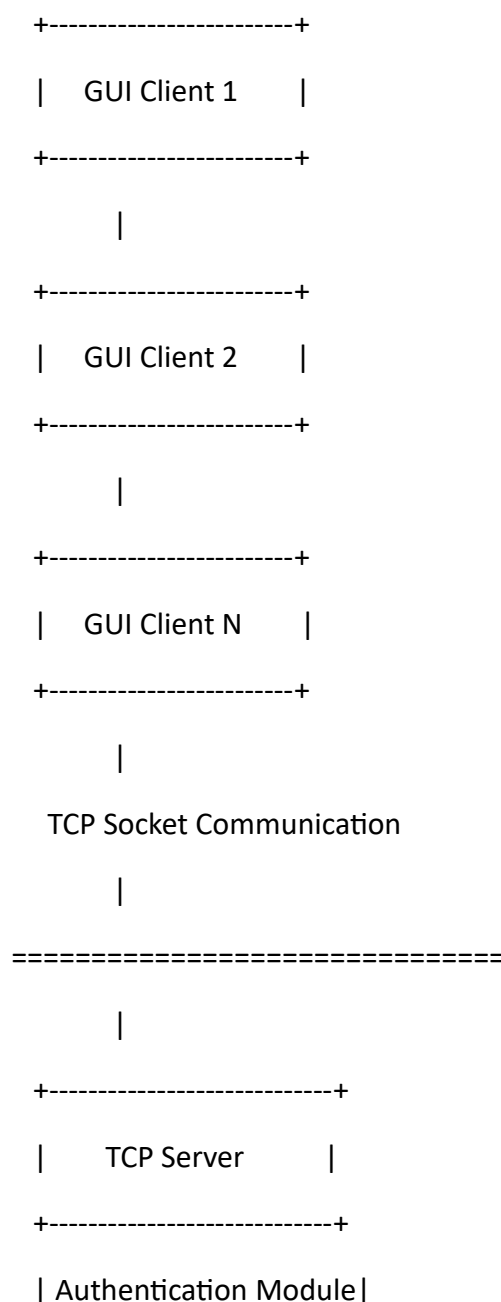
5.2 Hardware Requirements

Hardware	Specification
Processor:	Intel Core i3 or Higher
RAM:	Minimum 4 GB
Storage:	20 GB Free Space
OS:	Windows 10/11 with WSL Ubuntu
Network:	Virtual Network using Mininet

6. PROJECT ARCHITECTURE

The Secure SyncChat application follows a **Client-Server Architecture**, where a centralized server manages authentication, communication, security, session handling, configuration management, and persistent data storage, while multiple GUI clients interact with the server using TCP socket communication.

The architecture ensures that all communication is routed through the server, enabling centralized authentication, message forwarding, user management, security auditing, and performance monitoring. The modular design improves maintainability and allows new features such as configuration management, scalability testing, and performance evaluation to be integrated without affecting the core communication functionality.



```

| Session Manager |
| Connection Manager|
| Broadcast Manager|
| Private Messaging|
| Online User Manager|
| Chat History Manager|
| Security Logger|
| Configuration Manager|
| Performance Monitor|

```

```

+-----+

```

```

/      |      |      \
/      |      |      \

```

users.csv chat_history.csv security_log.txt config.json

6.1 Client Module

Each client is developed using the Tkinter GUI library. The client is responsible for collecting user credentials, establishing secure TCP connections, displaying chat messages, handling user interactions, validating user input, and communicating reliably with the server.

The client module includes:

- Login Window
- Signup Window
- Chat Window
- Online Users Panel
- Password Strength Checker
- Session Timeout Handler
- Message Validation
- Logout Management
- Connection Status Monitoring

- Automatic Configuration Loading

The client reads the server IP address and port number from a centralized configuration file, allowing deployment in different network environments without modifying the application source code.

6.2 Server Module

The server acts as the central controller of the application. It continuously listens for incoming client connections, authenticates users, manages active sessions, routes messages, stores activity logs, and monitors system performance. Multi-threading enables the server to handle multiple concurrent clients efficiently while maintaining reliable communication.

Major responsibilities include:

- Accept client connections
 - Authenticate users
 - Prevent duplicate logins
 - Manage active user sessions
 - Broadcast public messages
 - Deliver private messages
 - Maintain online user list
 - Store chat history
 - Generate security logs
 - Handle client disconnections gracefully
 - Load application configuration
 - Monitor resource utilization during performance testing
-

6.3 Database and Configuration Module

Instead of using a relational database, lightweight CSV files and a centralized configuration file are employed to simplify deployment while maintaining persistent storage and flexible application management.

users.csv

Stores:

- Username
- SHA-256 Password Hash

chat_history.csv

Stores:

- Timestamp
- Message Type
- Sender
- Receiver
- Message Content

security_log.txt

Stores:

- Login Success
- Failed Login
- Duplicate Login
- Logout
- Session Timeout
- Account Creation

config.json

Stores:

- Server IP Address
- Server Port
- Buffer Size
- Maximum Client Limit
- Application Configuration Parameters

The configuration file allows network settings to be modified without changing the source code, improving maintainability and simplifying deployment across different environments.

7. DESIGN METHODOLOGY

The development of **SyncChat** followed a modular software design approach, where each major functionality was implemented as an independent module. This modular architecture simplified development, testing, debugging, maintenance, and future enhancements while ensuring that new features could be integrated without affecting the existing system.

The overall development process consisted of the following phases.

Phase 1: GUI Design

The first stage involved designing an intuitive graphical user interface using the Tkinter framework. Separate windows were developed for user registration, login, and chat communication. Particular emphasis was placed on usability, consistency, responsiveness, and providing a user-friendly interface for interacting with the chat server.

Phase 2: Client-Server Communication

A TCP socket connection was established between clients and the centralized server to enable reliable message exchange. Multi-threading was implemented to allow the server to handle multiple client connections simultaneously without blocking communication, ensuring smooth real-time interaction among users.

Phase 3: Authentication

A secure authentication mechanism was developed where new users register through the signup interface. User passwords are hashed using the SHA-256 cryptographic algorithm before being stored in the user database. During login, the entered password is hashed again and compared with the stored hash value, ensuring that plaintext passwords are never stored or transmitted.

Phase 4: Security Implementation

Multiple security mechanisms were integrated into the application to improve confidentiality, integrity, and access control. These include duplicate login prevention, login attempt restriction, password validation, session timeout monitoring, secure event logging, message validation, and online user management.

Phase 5: Reliability and Configuration Management

After implementing the core communication and security features, the application was enhanced to improve reliability and maintainability. Centralized configuration management was introduced using a configuration file to store network and application parameters, allowing deployment settings to be modified without changing the source code. Connection management and exception handling were also improved to ensure graceful handling of unexpected client disconnections while maintaining uninterrupted communication for active users.

Phase 6: Performance Optimization and Scalability Evaluation

The final stage focused on evaluating and optimizing the application's performance under different workloads. Benchmark testing was conducted by connecting multiple concurrent clients within a Mininet environment to assess the system's scalability and resource utilization. Performance metrics including average message delay, throughput, CPU utilization, and memory consumption were recorded and analyzed. The collected data was stored in a CSV file and visualized using graphs to evaluate the efficiency and scalability of the developed application.

Phase 7: Testing and Validation

The completed application was tested extensively using multiple clients connected through Mininet. Functional testing verified all implemented features, including authentication, secure messaging, logging, session management, and configuration handling. Wireshark was used to inspect TCP communication and verify correct packet transmission during connection establishment, message exchange, and connection termination. Performance benchmarking further validated the application's reliability, scalability, and efficient resource utilization under varying client loads.

8. FEATURES IMPLEMENTED

The Secure **SyncChat** application was developed using a modular client-server architecture that combines secure communication, graphical user interface design, authentication mechanisms, session management, and performance optimization into a single integrated platform. Throughout the development process, multiple networking, security, reliability, and scalability features were implemented to provide a secure, efficient, and user-friendly communication system.

The application supports authenticated multi-client communication over TCP sockets while incorporating modern software engineering practices such as centralized configuration management, graceful connection handling, performance monitoring, and scalability evaluation.

The major features implemented during this assignment are described below.

8.1 User Registration

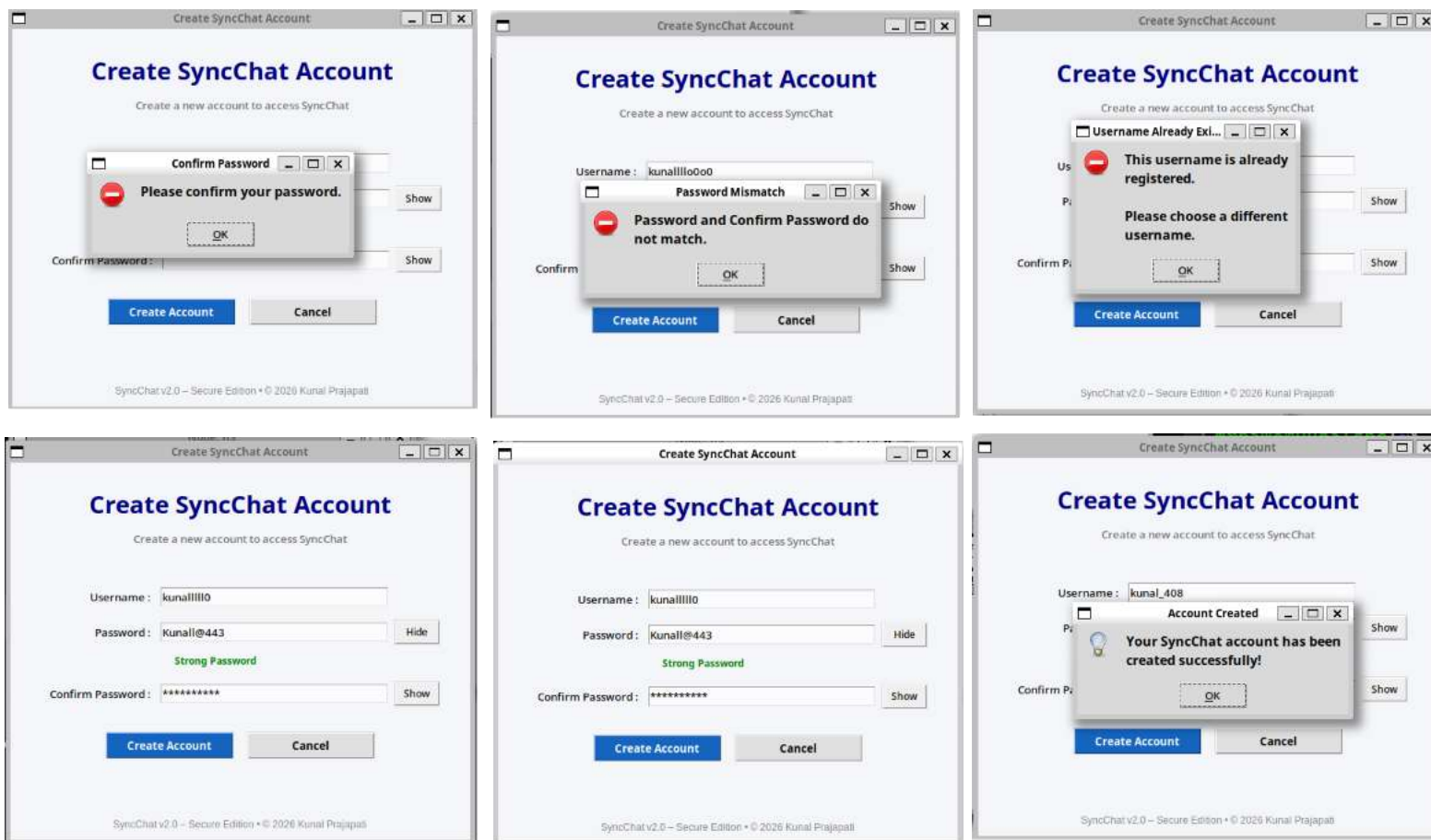
A graphical signup window was developed to allow new users to create accounts before accessing the chat application. The registration form collects the username, password, and confirmation password from the user. Before creating an account, the application performs multiple validation checks to ensure that only valid user information is stored.

The implemented validation mechanisms include:

- Username format validation
- Password strength verification
- Password confirmation verification
- Duplicate username detection
- Empty field validation

Only after successfully passing all validation checks is the user account created and stored in the database.





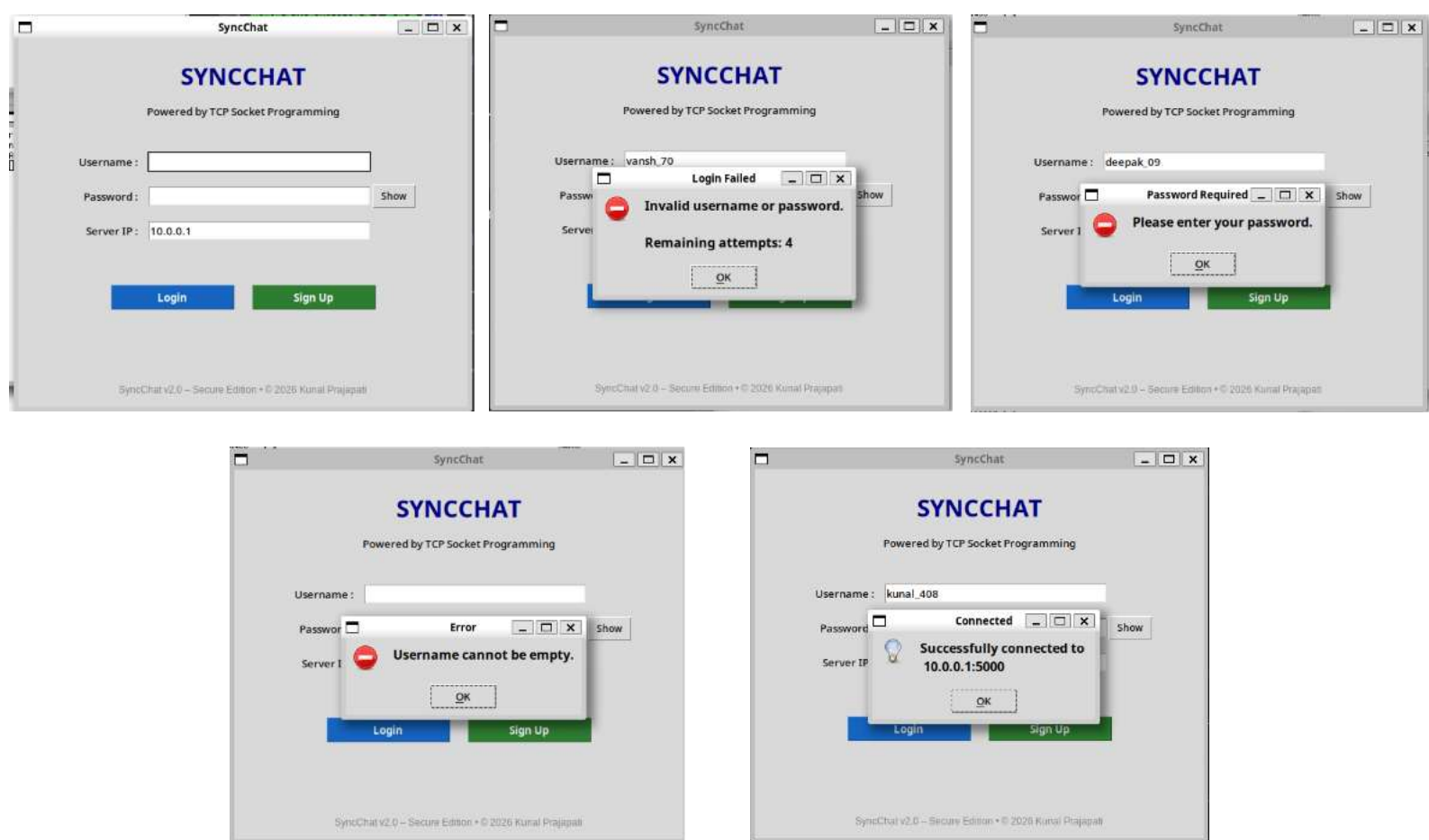
8.2 Secure Login System

The login module authenticates users before allowing access to the chat application. User credentials are compared against the securely stored password hashes in the users.csv database. If the entered credentials match the stored records, the user is successfully connected to the server.

The login system also includes multiple security mechanisms to prevent unauthorized access.

Implemented features include:

- Username validation
- Password validation
- Authentication using SHA-256
- Invalid login detection
- Login success confirmation



8.3 Duplicate Login Prevention

To ensure that a single user account cannot be used simultaneously from multiple client instances, the server checks whether a username is already active before completing the authentication process.

If another active session already exists with the same username, the login request is rejected, and an appropriate message is displayed to the user.

This mechanism prevents identity duplication and improves overall application security.



8.4 Login Attempt Restriction

To reduce the possibility of brute-force password guessing attacks, the application limits the number of consecutive unsuccessful login attempts.

If the user enters an incorrect password five times consecutively, further login attempts are temporarily blocked for 30 seconds. After the timeout period expires, authentication becomes available again.

This mechanism significantly increases the security of user accounts.



8.5 Password Hashing

One of the most important security enhancements introduced in this assignment is secure password storage.

Instead of saving passwords in plain text, every password is converted into a SHA-256 cryptographic hash before being stored inside the users.csv file.

During login, the entered password is hashed again and compared with the stored hash value. Since only hash values are stored, the original password cannot be recovered from the database.

This mechanism protects user credentials even if the database file is accessed by unauthorized individuals.

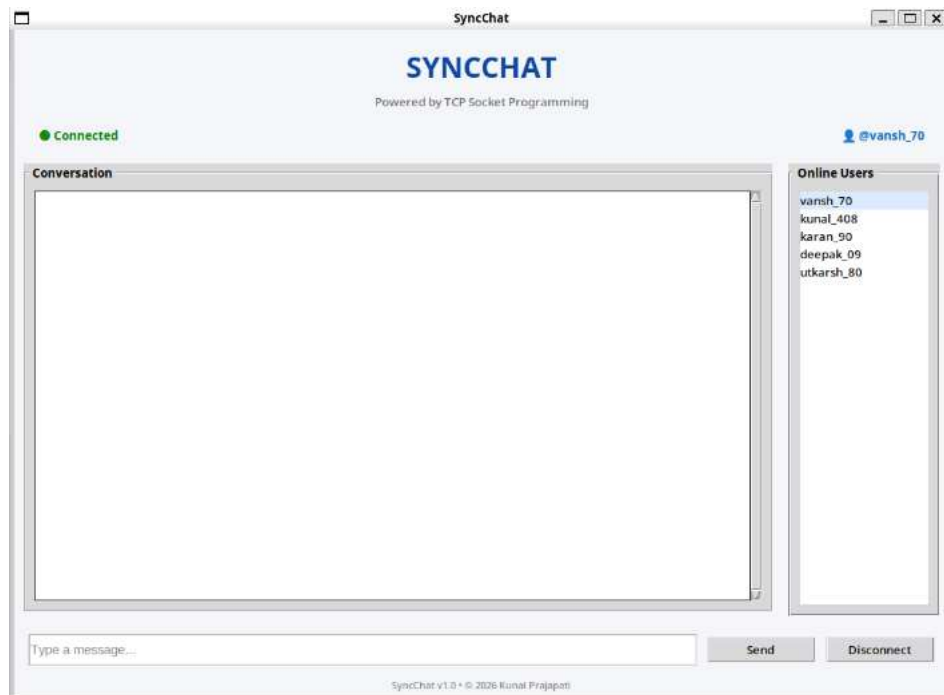
```
mrbaue@kunalapc: ~ - X  mrbaue@kunalapc: ~ - X  +  v  -  □  X
mrbaue@kunalapc:~/UU2409000396_Kunal_Prajapati_GUI_chatserver_assignment6$ cat users.csv
username,password_hash
karan_90,ae6bfa924974e243916b55f56dd3476059be89c4902539702aa0ed4d7b11bad2
deepak_09,edd0e82c293d8eb0c123b631d8d6681281926027fe746906d0d5c39f181ddb14
kunal_408,404e71d3382b62ec938408e90a47105566dd972a93bfdb48b351da7de4ec641
utkarsh_80,415bfe27259b24d91d6db6d05fa23ada368e17788edc14fe2e4aead6f4a88ac1
vansh_70,82c95c9eb8d1caa4dc6d82a8a6f8cbb58c01f80b8bfe705a1db5d98909c0b7af
mrbaue@kunalapc:~/UU2409000396_Kunal_Prajapati_GUI_chatserver_assignment6$ |
```


8.6 Online User Management

The server continuously maintains a list of all active users connected to the chat server.

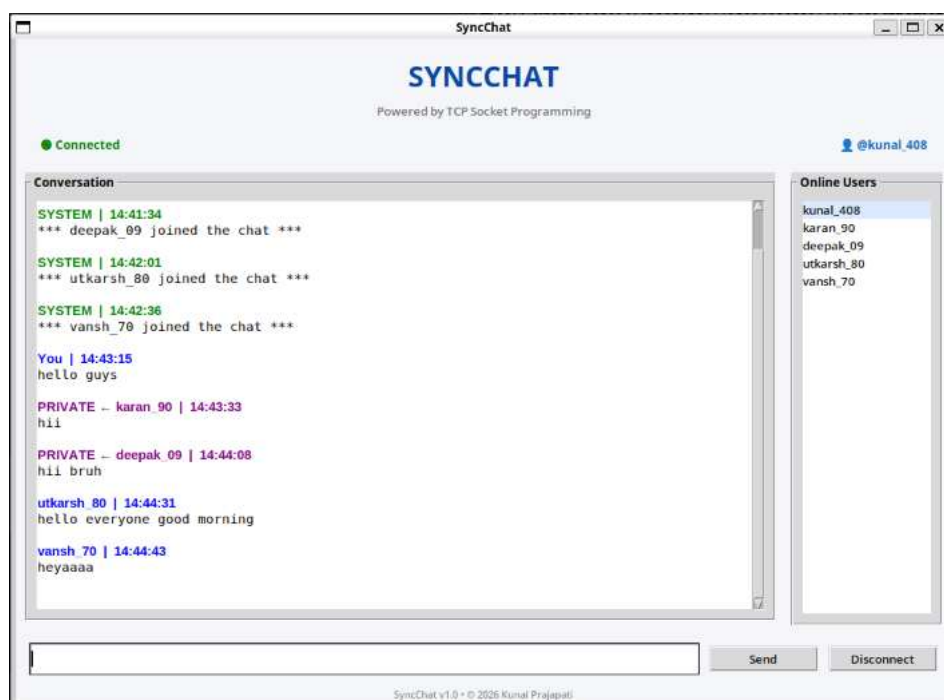
Whenever a user logs in or disconnects from the application, the updated user list is immediately transmitted to every connected client.

The online users panel allows users to view currently active participants in real time.



8.7 Broadcast Messaging

Broadcast messaging enables a user to send a message to every connected client simultaneously.



Whenever a broadcast message is transmitted, the server receives the message and forwards it to all connected clients except the sender where appropriate. This ensures real-time communication among all participants.

Broadcast messaging forms the primary communication mechanism of the application.

8.8 Private Messaging

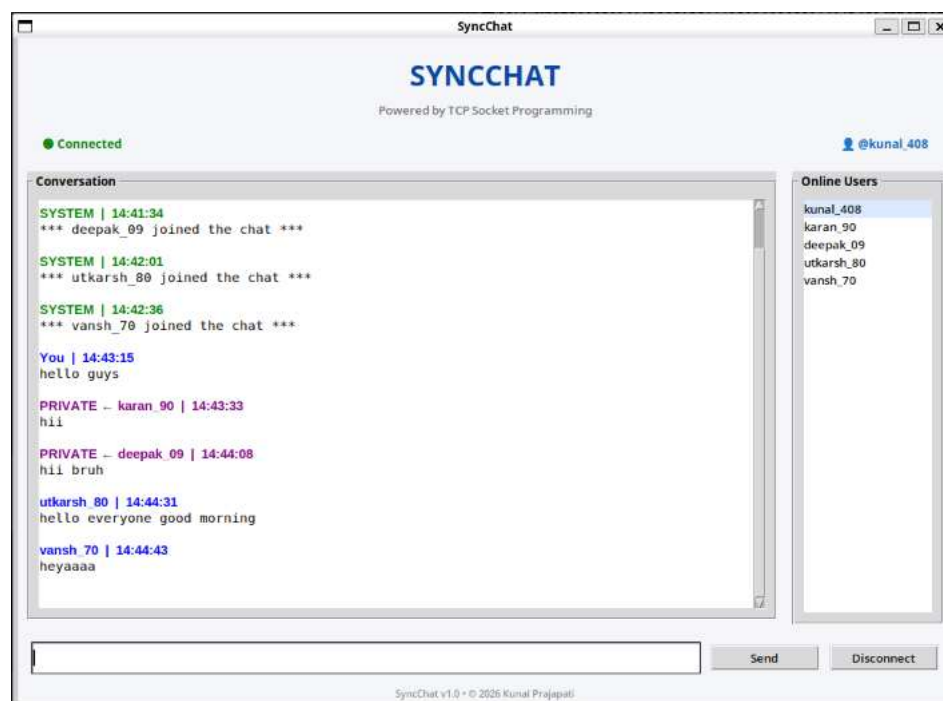
In addition to broadcast communication, the application supports secure private messaging.

Users can communicate directly with a selected participant using the following command:

`/msg username message`

The server identifies the intended recipient and delivers the message only to that user while simultaneously displaying a confirmation message to the sender.

Private messaging demonstrates selective communication over the TCP connection.

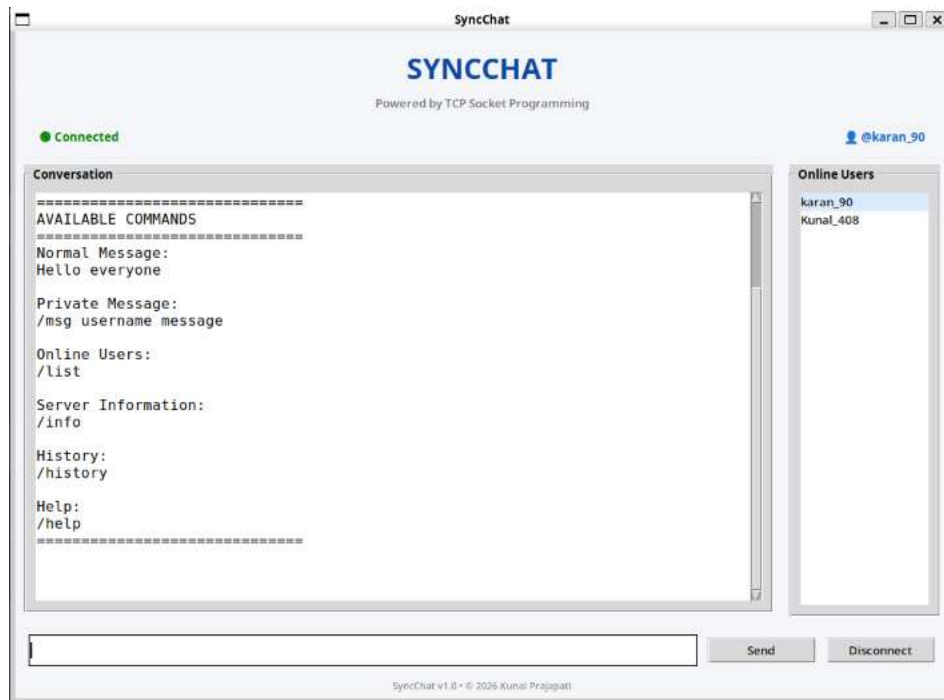


8.9 Chat Commands

Several built-in commands were implemented to improve user interaction with the server.

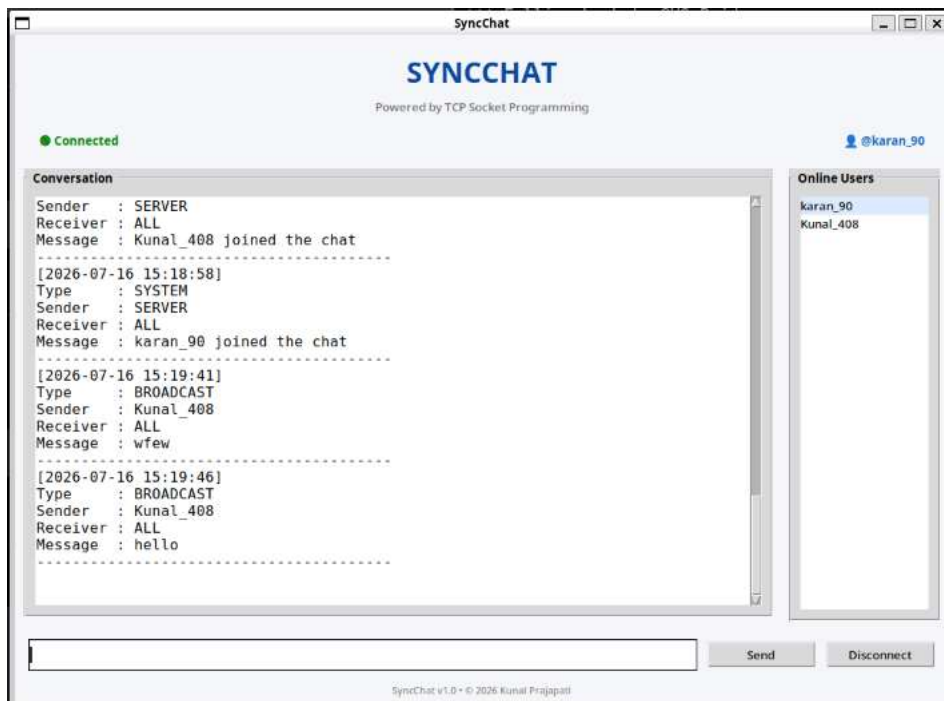
`/help`

Displays all available commands supported by the server.



/history

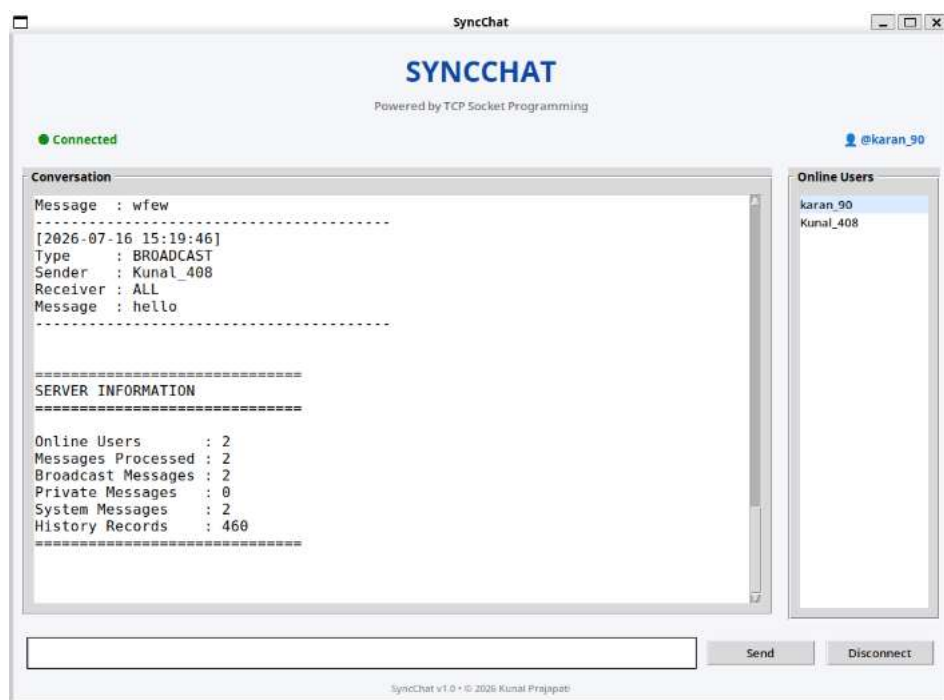
Displays the most recent chat messages stored inside chat_history.csv.



/info

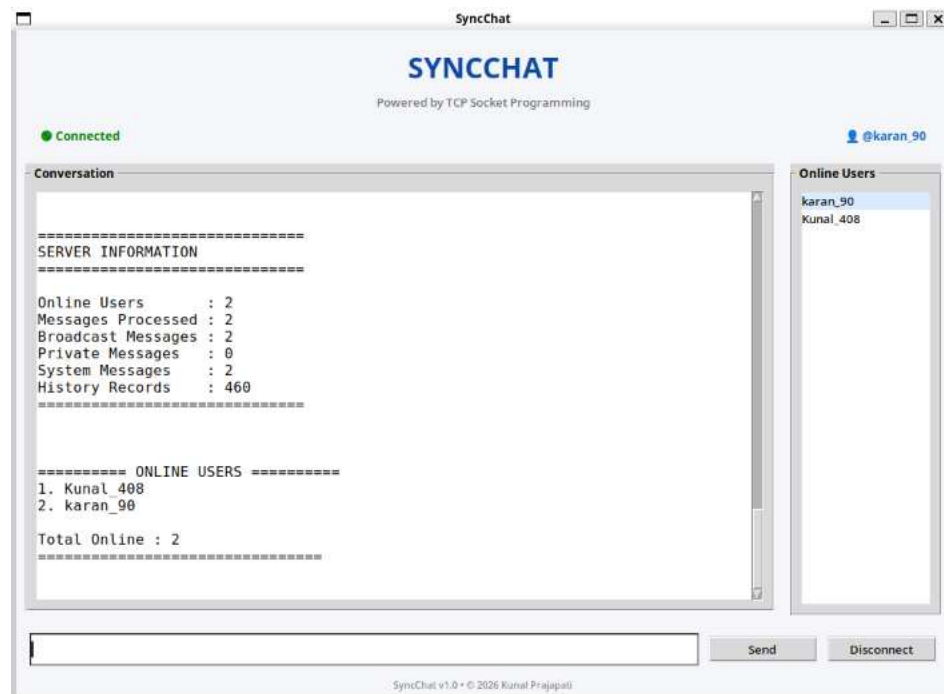
Displays useful server statistics such as:

- Connected users
- Messages processed
- Broadcast messages
- Private messages
- Chat history records



/list

Displays all currently connected users.



8.10 Message Length Validation

To prevent excessively long messages from affecting communication, a maximum message length was defined.

Whenever a user attempts to send a message exceeding the specified limit, the application displays an error message and prevents the message from being transmitted.

This validation improves usability while protecting the application against oversized user input.



8.11 Connection Management

Reliable connection management was implemented to improve the stability of communication between clients and the server. The server continuously monitors active client connections and gracefully handles unexpected client disconnections without interrupting communication for other connected users. Whenever a client disconnects, the corresponding session is removed from the active user list, associated resources are released, and the remaining clients continue communicating without requiring a server restart.

Implemented features include:

- Graceful client disconnection handling
- Automatic cleanup of inactive client sessions
- Stable multi-threaded connection management
- Continuous server availability during client failures

8.12 Reliability Enhancement

Several reliability improvements were introduced to ensure stable operation during extended execution and multi-client communication. Exception handling mechanisms were incorporated throughout the client and server modules to prevent unexpected runtime failures. Proper resource cleanup and controlled socket termination help maintain application stability even when network interruptions occur.

Implemented features include:

- Improved exception handling
 - Graceful socket shutdown
 - Resource cleanup
 - Stable server execution
 - Fault-tolerant communication
-

8.13 Configuration Management

A centralized configuration file (config.json) was introduced to store application parameters such as server IP address, port number, buffer size, and maximum client limits. This eliminates the need to modify source code whenever deployment settings change and improves maintainability by separating configuration from implementation.

Configuration parameters include:

- Server IP address
 - Server port
 - Buffer size
 - Maximum client limit
 - Application settings
-

8.14 Scalability Enhancement

The application was evaluated under increasing client workloads to verify its ability to support concurrent communication. Benchmark testing was performed using multiple clients connected through a Mininet virtual network. The server successfully maintained reliable communication while handling increasing numbers of simultaneous users.

Scalability features include:

- Multi-threaded client handling

- Concurrent user support
 - Efficient message routing
 - Stable communication under increasing workloads
 - Successful testing with multiple concurrent clients
-

8.15 Performance Evaluation

A dedicated performance benchmarking module was developed to evaluate the efficiency of the application under different workloads. The benchmarking process measured average message delay, throughput, CPU utilization, and memory consumption while multiple clients exchanged messages simultaneously. The collected results were stored in a CSV file and visualized using graphical analysis to evaluate the application's performance and scalability.

Performance metrics include:

- Average message delay
- Message throughput
- CPU utilization
- Memory consumption
- Performance graph generation
- Automated benchmark reporting

9. GUI DESIGN

A user-friendly Graphical User Interface (GUI) was developed using Python's Tkinter library to provide a simple, secure, and interactive communication platform between users and the chat server. Unlike traditional command-line chat applications, the developed interface enables users to perform all operations through graphical controls, making the application easier to understand, operate, and manage.

The GUI consists of three primary interfaces: the Login Window, Signup Window, and Chat Window. Each interface has been designed to perform a specific function while maintaining a consistent appearance, responsive user interaction, and seamless communication with the server. The graphical interface also integrates securely with the application's authentication system, centralized configuration management, and real-time communication modules, providing a stable and user-friendly environment for multi-client communication.

9.1 Login Window

The Login Window serves as the entry point of the application. It authenticates registered users before granting access to the chat system. The interface includes input fields for Username, Password, and Server IP Address, along with Login and Sign Up buttons.

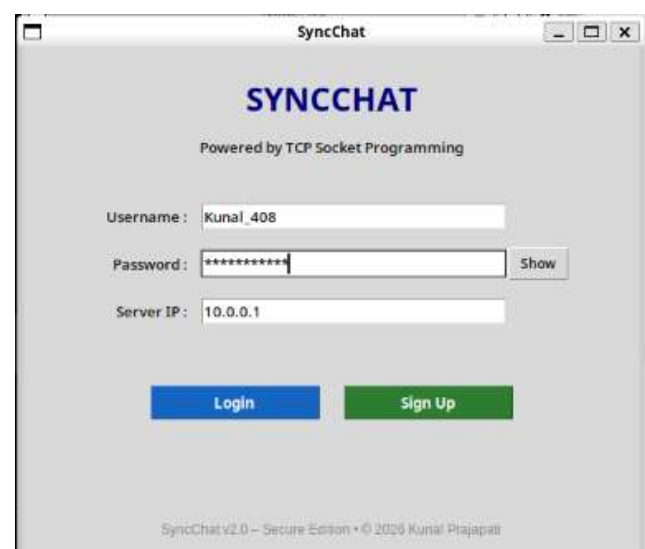
Additional usability features such as password show/hide functionality have also been incorporated to improve user convenience while entering credentials.

The Login Window performs multiple validation checks before establishing a connection with the server. Invalid credentials or empty input fields generate appropriate error messages to prevent unauthorized access.

The Login Window also supports configurable server connectivity through a centralized configuration file. The server IP address and communication parameters can be modified without changing the application source code, improving deployment flexibility across different network environments. Connection status is continuously monitored to provide immediate feedback whenever the client successfully connects to or disconnects from the server.

Functions of the Login Window:

- Accept user credentials.
- Connect to the TCP server.
- Authenticate registered users.
- Validate input fields.
- Display login status.
- Redirect new users to the Signup Window.



9.2 Signup Window

The Signup Window allows new users to register themselves before accessing the chat application.

The interface was redesigned to maintain visual consistency with the Login Window while providing additional security features. The registration form consists of Username, Password, and Confirm Password fields together with password visibility controls.

Several validation mechanisms are executed before account creation.

These include:

- Username format validation.
- Password strength evaluation.
- Duplicate username detection.
- Password confirmation verification.
- Empty field validation.

Only after successful validation is the account stored securely inside users.csv.

Functions of the Signup Window:

- Register new users.
- Validate usernames.
- Validate passwords.
- Prevent duplicate usernames.
- Evaluate password strength.
- Securely store credentials.



9.3 Chat Window

After successful authentication, the user is redirected to the Chat Window.

The Chat Window acts as the primary communication interface of the application and provides several interactive components for real-time messaging.

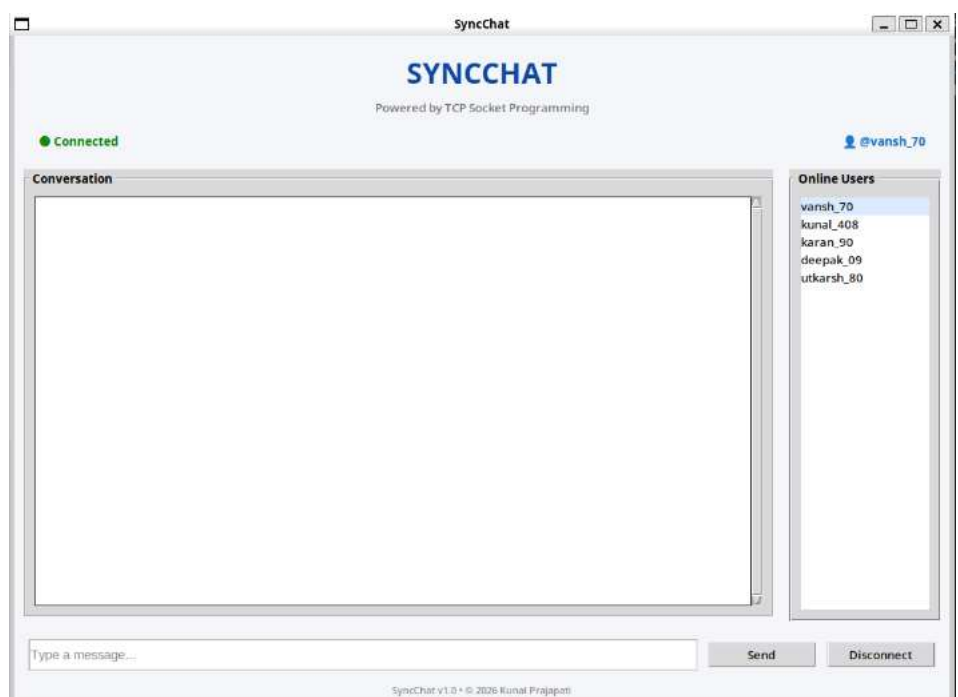
The interface includes:

- Conversation Area
- Message Input Box
- Send Button
- Disconnect Button
- Online Users Panel
- Connection Status
- Application Footer

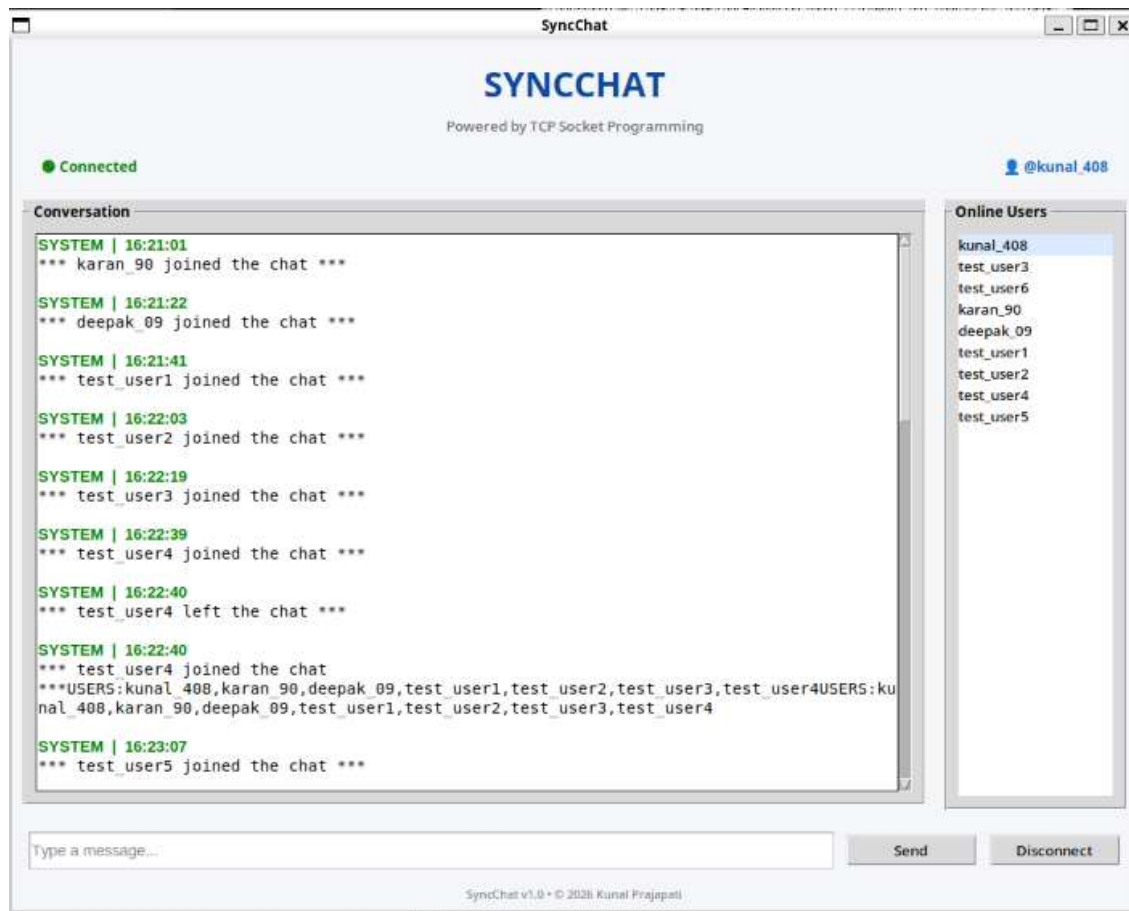
Users can send normal broadcast messages as well as private messages using supported commands. The Online Users panel dynamically updates whenever users join or leave the chat session.

Functions of the Chat Window:

- Broadcast communication.
- Private messaging.
- Online user monitoring.
- Session timeout monitoring.
- Secure logout.
- Display incoming messages.



The Chat Window was further enhanced to support reliable communication during multi-client operation. Active user sessions are maintained dynamically, and the interface remains responsive even when multiple users exchange messages simultaneously. The client also displays connection status information, allowing users to identify successful server connections and unexpected disconnections during execution.



9.4 GUI Validation Messages

The application displays informative dialog boxes whenever user input violates predefined validation rules.

Validation dialogs improve usability by informing users about the exact cause of an error while preventing invalid operations.

Examples include:

- Username Required
- Password Required
- Invalid Username
- Username Already Exists

- Password Mismatch
- Duplicate Login
- Too Many Login Attempts
- Session Timeout
- Message Length Exceeded
- Connection Failed
- Server Unreachable

These dialog boxes significantly improve user experience by providing immediate feedback.

9.5 GUI Enhancements

Compared to the previous assignment, several improvements were incorporated into the graphical interface.

These improvements include:

- Redesigned Signup Window.
- Consistent interface design.
- Password visibility toggle.
- Password strength indicator.
- Better button styling.
- Improved spacing and alignment.
- Footer displaying application version.
- Online user list.
- Connection status indicator.
- Enhanced validation dialogs.
- Configuration-based server connection.
- Improved connection status monitoring.
- Better stability during multi-client communication.
- Support for scalable client-server deployment.

These enhancements provide a more professional appearance while improving overall usability.

10. SECURITY MECHANISMS

Security is one of the fundamental components of the Secure SyncChat application. The system integrates multiple cybersecurity mechanisms to ensure the confidentiality, integrity, and availability of user communication while protecting authentication data and maintaining secure client-server interactions. Various preventive and protective measures have been implemented to safeguard user accounts, monitor system activities, and improve the overall reliability of the application. The implemented security mechanisms are described below.

10.1 User Authentication

Authentication ensures that only registered users are allowed to access the chat server.

Whenever a client attempts to connect, the server verifies the submitted username and password against the stored credentials inside users.csv.

Only authenticated users are granted access to the chat interface.

Authentication prevents unauthorized users from joining the communication system.

10.2 Password Hashing

One of the most significant security improvements is the implementation of SHA-256 password hashing.

During account registration, the entered password is converted into a fixed-length SHA-256 hash before being stored.

Example:

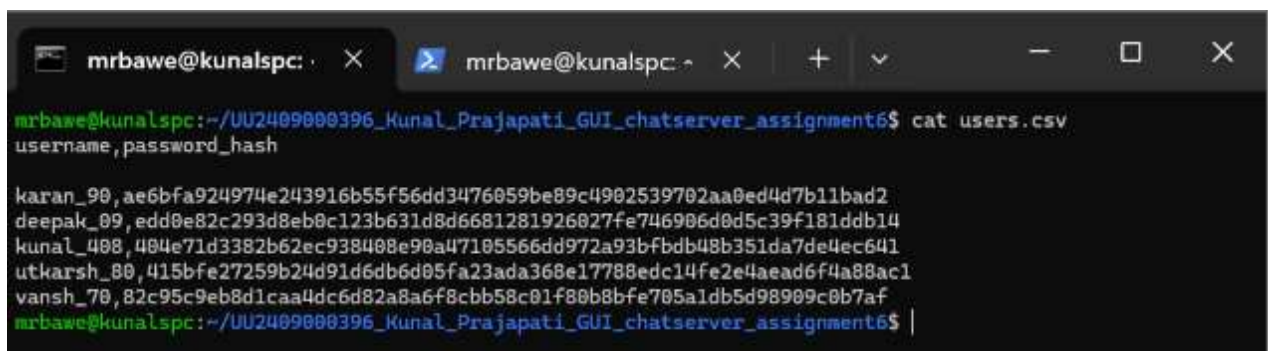
Original Password: MyPassword123

↓

SHA-256 Hash: a8f5f167f44f4964e6c998dee827110c...

Because only the hash value is stored, the original password cannot be recovered from the database.

This significantly improves credential security.

A terminal window screenshot showing a user named 'mrbaawe' at a host 'kunal' with IP '2409080396'. The user is in a directory 'Kunal_Prajapati_GUI_chatserver_assignment6'. They run the command 'cat users.csv'. The output shows a CSV file with two columns: 'username' and 'password_hash'. The first row contains 'karan_90' and a long SHA-256 hash. The second row contains 'deepak_09' and another long SHA-256 hash. The third row contains 'kunal_408' and a third long SHA-256 hash. The fourth row contains 'utkarsh_80' and a fourth long SHA-256 hash. The fifth row contains 'vansh_70' and a fifth long SHA-256 hash. The terminal prompt is 'mrbaawe@kunal\$'.

10.3 Duplicate Username Prevention

During registration, the application verifies whether the entered username already exists inside the user database.

If a duplicate username is detected, account creation is immediately rejected.

This ensures that every registered user has a unique identity.



10.4 Duplicate Login Prevention

Even after successful registration, users are prevented from logging into multiple sessions simultaneously.

Whenever a second login request is received using an already active username, the server rejects the request and displays an error dialog.

This prevents identity duplication and session misuse.



10.5 Login Attempt Restriction

To protect user accounts against brute-force attacks, the application limits the number of unsuccessful login attempts.

After five consecutive incorrect password entries, further login requests are blocked for thirty seconds.

This mechanism greatly reduces the possibility of password guessing attacks.



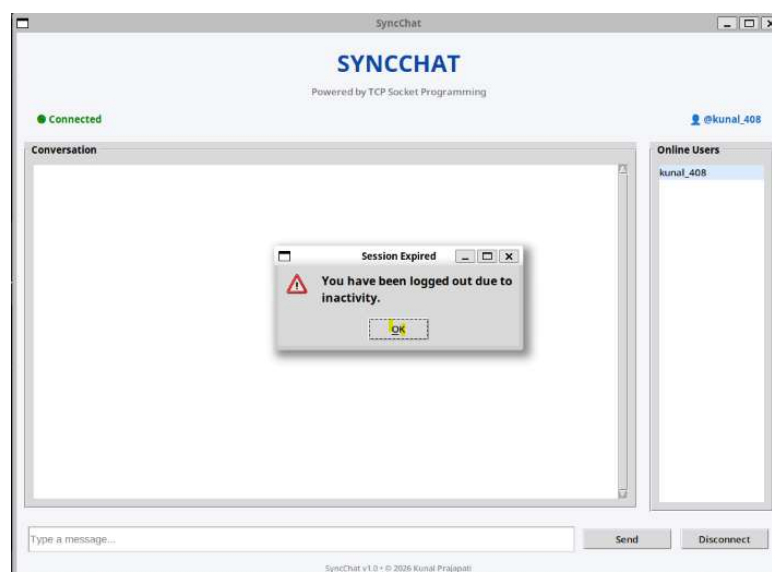
10.6 Session Timeout

The application continuously monitors user activity.

Whenever the user remains inactive beyond the predefined timeout period, the current session is automatically terminated.

This prevents unauthorized access when users leave their systems unattended.

The event is also recorded inside the security log.



10.7 Message Length Validation

To improve stability and prevent oversized network packets, outgoing messages are validated before transmission.

Messages exceeding the predefined maximum length are rejected and an informative dialog box is displayed.

This prevents accidental flooding and maintains application responsiveness.



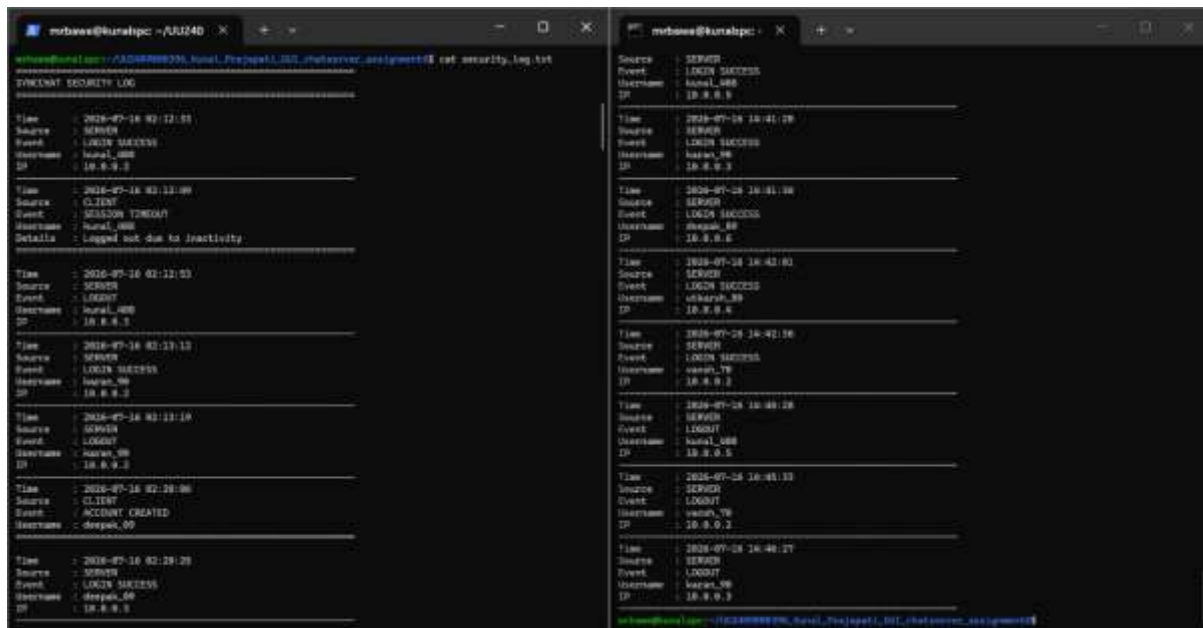
10.8 Secure Logging

The application maintains a dedicated security log separate from normal chat history.

Important security events recorded include:

- Account Creation
- Login Success
- Logout
- Session Timeout
- Duplicate Login Attempt
- Failed Login Attempt

These records improve accountability and simplify future security audits.



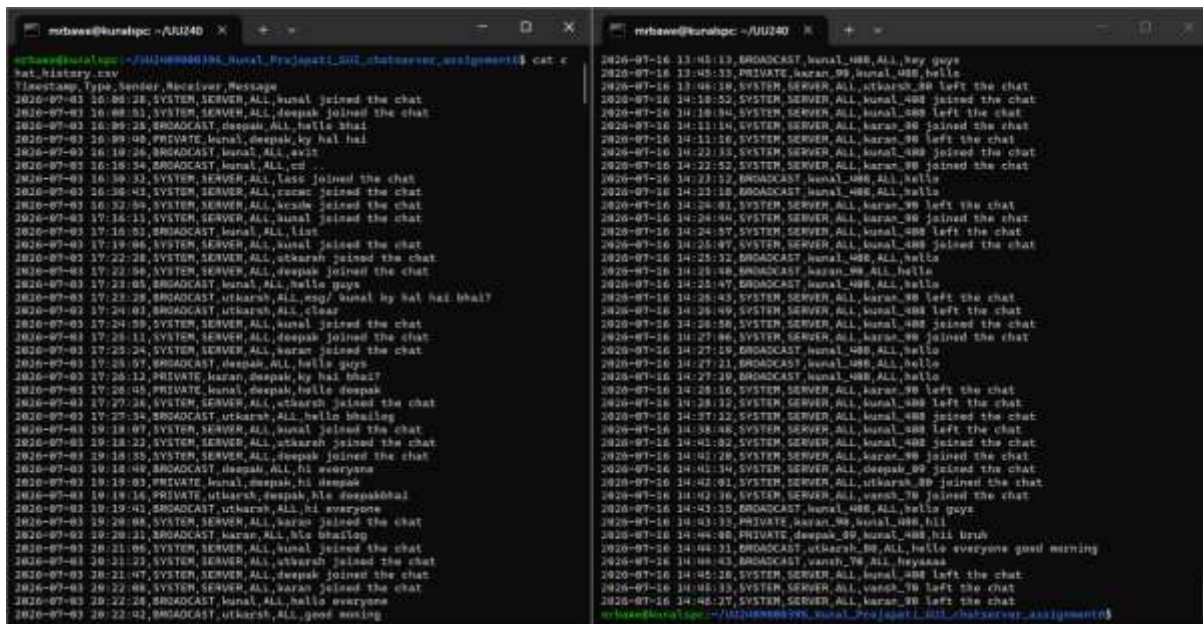
10.9 Chat History

Every successfully transmitted message is recorded inside chat_history.csv.

Each record contains:

- Timestamp
- Message Type
- Sender
- Receiver
- Message Content

Persistent chat history allows previous communication to be reviewed even after restarting the application.

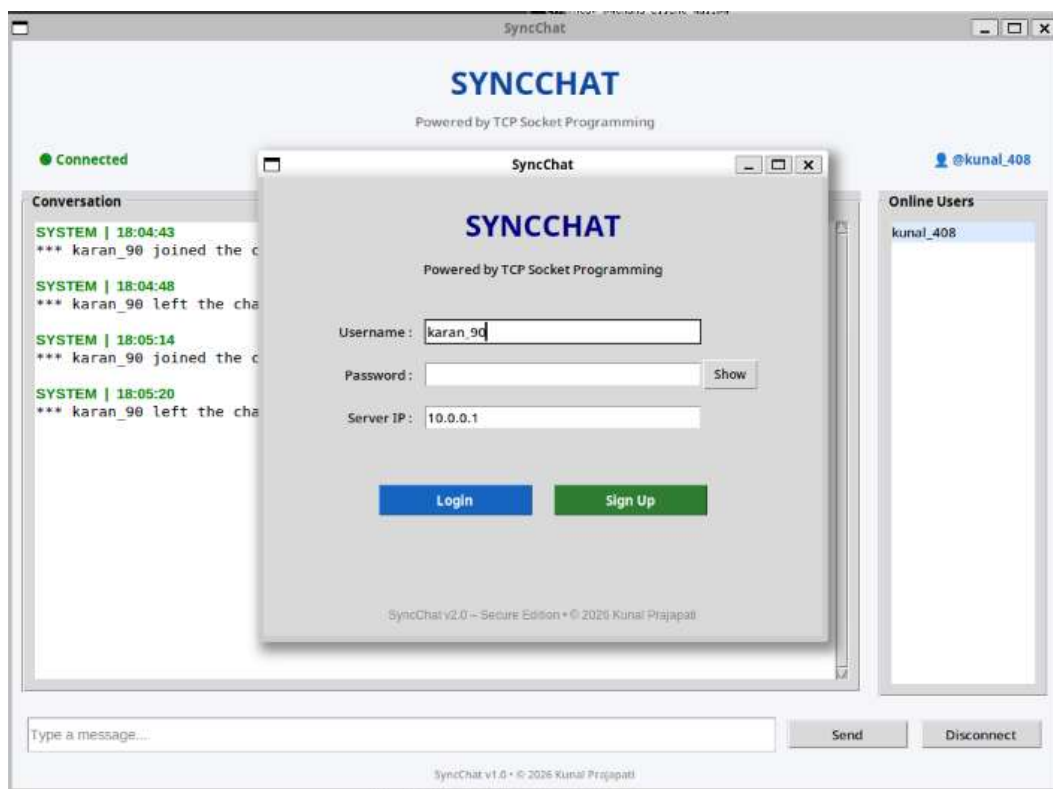


10.10 Connection Reliability

Reliable communication is essential for maintaining secure client-server interactions. The application incorporates improved connection management to detect unexpected client disconnections and release associated resources without affecting other active users. Proper exception handling ensures that communication continues uninterrupted for remaining connected clients, thereby improving both system stability and reliability.

Implemented mechanisms include:

- Graceful client disconnection handling
- Automatic session cleanup
- Socket exception handling
- Continuous server availability
- Reliable multi-client communication

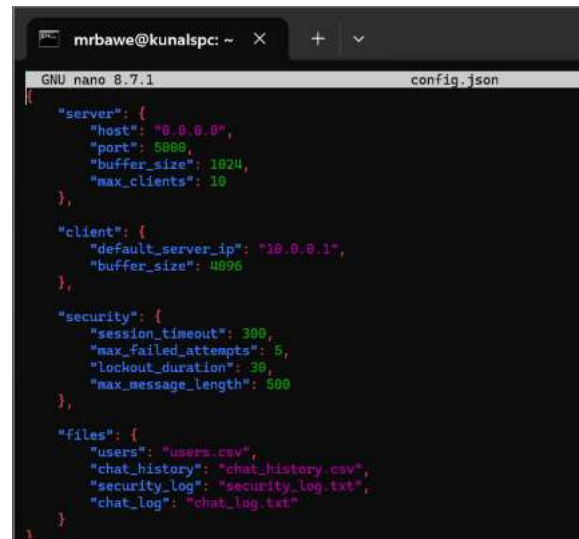


10.11 Configuration Security

The application employs centralized configuration management through a dedicated configuration file containing server communication parameters. Separating configuration settings from the application source code improves maintainability while reducing the possibility of configuration errors during deployment. Centralized configuration also simplifies system administration across different network environments.

The configuration file stores:

- Server IP Address
- Server Port Number
- Buffer Size
- Maximum Client Connections
- Application Parameters



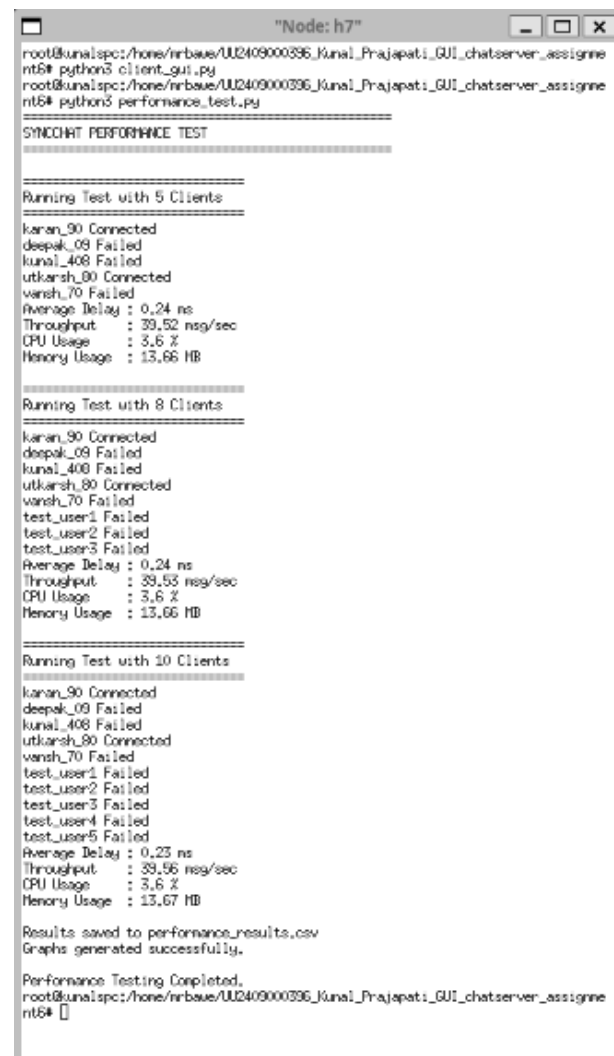
```
GNU nano 8.7.1 config.json
{
  "server": {
    "host": "0.0.0.0",
    "port": 5000,
    "buffer_size": 1024,
    "max_clients": 10
  },
  "client": {
    "default_server_ip": "10.0.0.1",
    "buffer_size": 4096
  },
  "security": {
    "session_timeout": 300,
    "max_failed_attempts": 5,
    "lockout_duration": 30,
    "max_message_length": 500
  },
  "files": {
    "users": "users.csv",
    "chat_history": "chat_history.csv",
    "security_log": "security_log.txt",
    "chat_log": "chat_log.txt"
  }
}
```

10.12 Security Monitoring During Performance Evaluation

Security mechanisms were verified during multi-client performance evaluation to ensure that authentication, message transmission, session management, and logging remained functional under increasing workloads. Testing confirmed that the implemented security controls continued operating correctly while multiple concurrent clients communicated with the server, demonstrating that system security was maintained alongside application scalability.

The evaluation verified:

- User authentication
- Session management
- Secure logging
- Reliable message delivery
- Stable operation under concurrent client loads



```
root@kunal: /home/nrbaue/U2409000396_Kunal_Prajapati_GUI_chatserver_assignment
nt6# python3 client_gui.py
root@kunal: /home/nrbaue/U2409000396_Kunal_Prajapati_GUI_chatserver_assignment
nt6# python3 performance_test.py

SYNCHRONOUS PERFORMANCE TEST

Running Test with 5 Clients

karan_90 Connected
deepak_09 Failed
kunal_408 Failed
utkarsh_80 Connected
varsh_70 Failed
Average Delay : 0.24 ns
Throughput : 39.52 msg/sec
CPU Usage : 3.6 %
Memory Usage : 13.66 MB

Running Test with 8 Clients

karan_90 Connected
deepak_09 Failed
kunal_408 Failed
utkarsh_80 Connected
varsh_70 Failed
test_user1 Failed
test_user2 Failed
test_user3 Failed
Average Delay : 0.24 ns
Throughput : 39.52 msg/sec
CPU Usage : 3.6 %
Memory Usage : 13.66 MB

Running Test with 10 Clients

karan_90 Connected
deepak_09 Failed
kunal_408 Failed
utkarsh_80 Connected
varsh_70 Failed
test_user1 Failed
test_user2 Failed
test_user3 Failed
test_user4 Failed
test_user5 Failed
Average Delay : 0.23 ns
Throughput : 39.56 msg/sec
CPU Usage : 3.6 %
Memory Usage : 13.67 MB

Results saved to performance_results.csv
Graphs generated successfully.

Performance Testing Completed.
root@kunal: /home/nrbaue/U2409000396_Kunal_Prajapati_GUI_chatserver_assignment
nt6#
```

11. EXPERIMENTAL RESULTS AND TESTING

After implementing the networking, security, reliability, and performance optimization features, the Secure SyncChat application was thoroughly tested in a Mininet-based virtual network environment. Multiple client instances were connected to a centralized TCP server to evaluate authentication, secure communication, session handling, logging, configuration management, scalability, and overall system performance.

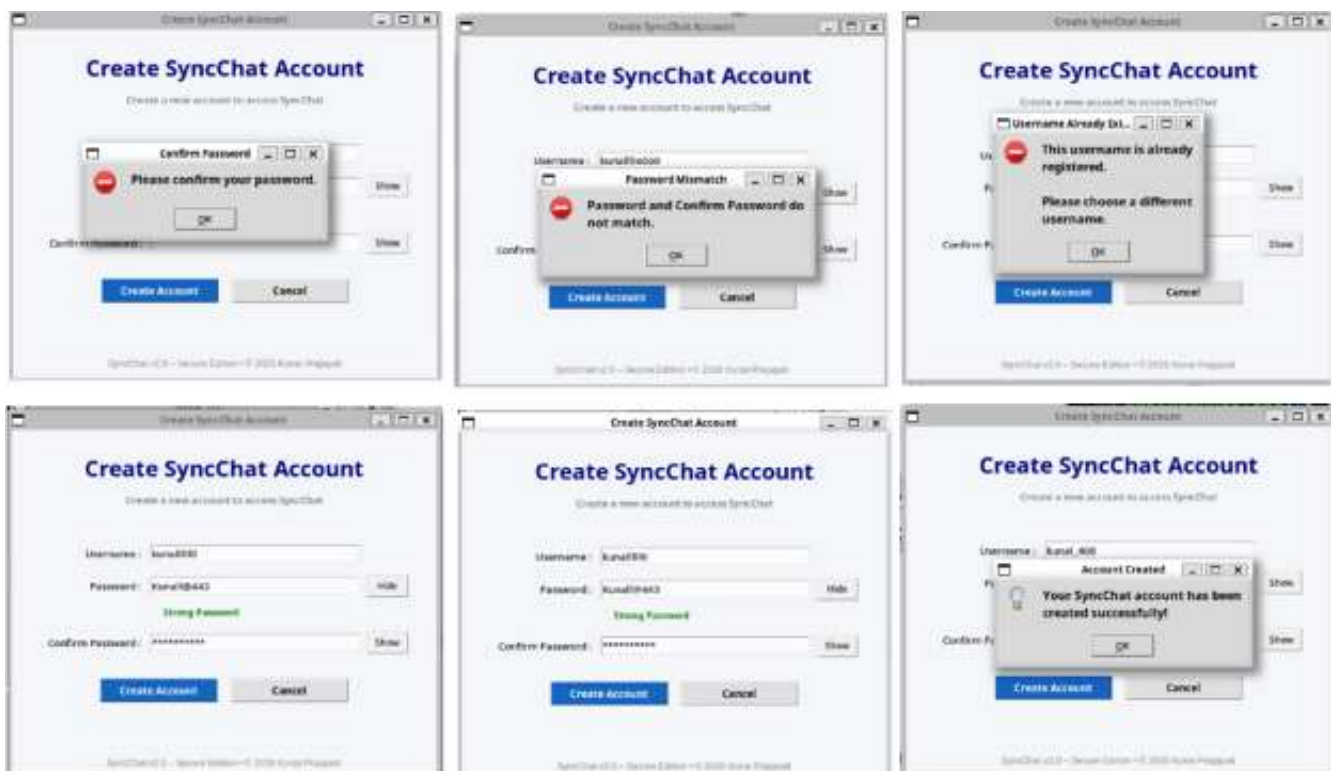
Each module was tested individually before performing integrated system testing. In addition to functional verification, benchmark testing was conducted under different client loads to evaluate throughput, message delay, CPU utilization, memory consumption, and system reliability. The experimental results confirmed that all implemented modules functioned according to the expected design while maintaining stable performance under increasing workloads.

11.1 User Registration Testing

The registration module was tested by creating multiple user accounts using different combinations of valid and invalid inputs.

During testing, the application successfully accepted valid usernames and securely stored their SHA-256 hashed passwords inside the **users.csv** database.

Various validation conditions such as duplicate usernames, invalid username formats, password mismatch, and empty fields were also tested successfully.



Observation

- Valid accounts were created successfully.
 - Duplicate usernames were rejected.
 - Password mismatch generated an error.
 - Weak passwords displayed appropriate strength indicators.
 - Passwords were never stored in plain text.
-

11.2 User Authentication Testing

The login module was tested using both valid and invalid credentials.

Registered users successfully entered the chat interface after authentication, whereas incorrect credentials generated informative error dialogs.

Multiple consecutive incorrect password attempts activated the temporary login restriction



mechanism.

Observation

- Correct credentials authenticated successfully.

- Incorrect credentials were rejected.
- Login attempt restriction worked correctly.
- Authentication time remained very small.

11.3 Duplicate Login Testing

The duplicate login prevention mechanism was verified by attempting to log in simultaneously using the same username from multiple clients.

The first client successfully entered the chat room, while the second login request was immediately rejected by the server.

This confirmed that the server correctly maintained active session information.

Observation

- Duplicate sessions were prevented.
- Existing user session remained unaffected.
- Appropriate warning message displayed.



11.4 Chat Communication Testing

Broadcast and private messaging functionalities were evaluated by connecting multiple clients simultaneously.

Broadcast messages were successfully delivered to every connected user, whereas private messages reached only the intended recipient.

No message loss or noticeable transmission delay was observed during testing.

Observation

- Broadcast communication successful.

- Private communication successful.
- Message ordering preserved.
- No duplicate packets observed.



11.5 Online User Management Testing

Whenever a user joined or left the server, the online user list was updated automatically.

The server successfully synchronized the list across all active clients.



Observation

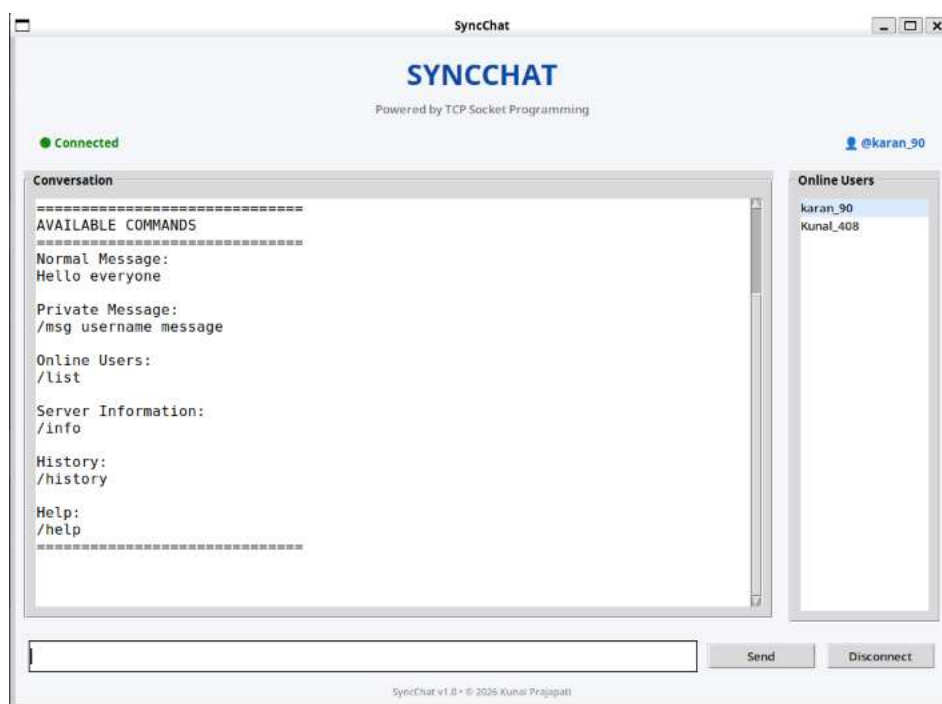
- Online users updated correctly.
 - User disconnection reflected immediately.
 - Session timeout also removed inactive users.
-

11.6 Server Command Testing

Several built-in commands were tested to verify server-side functionality.

Help Command

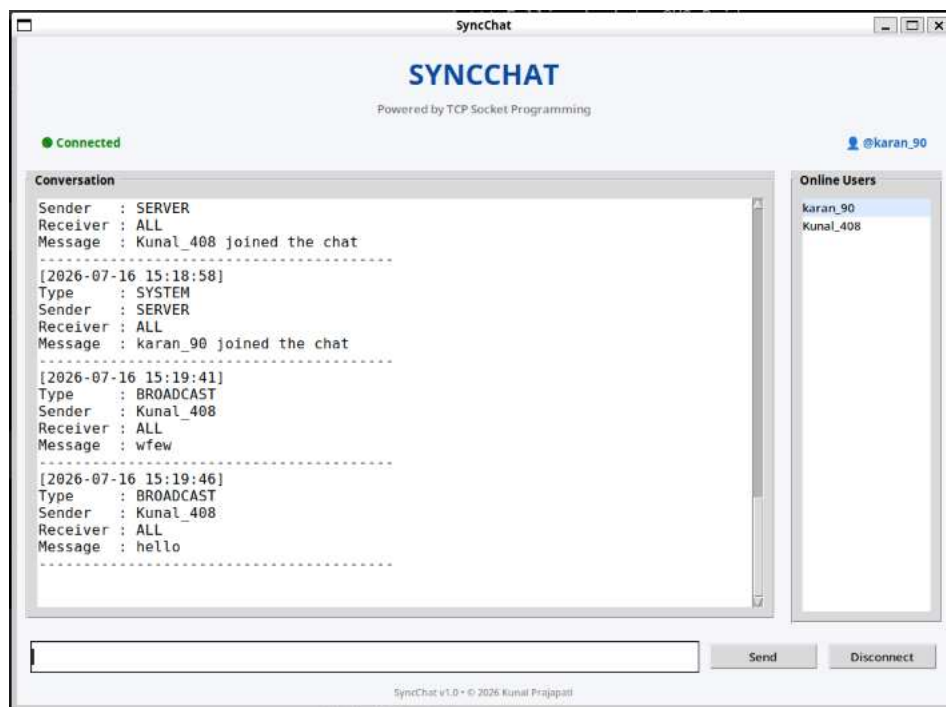
The **/help** command displayed all supported commands available to the user.



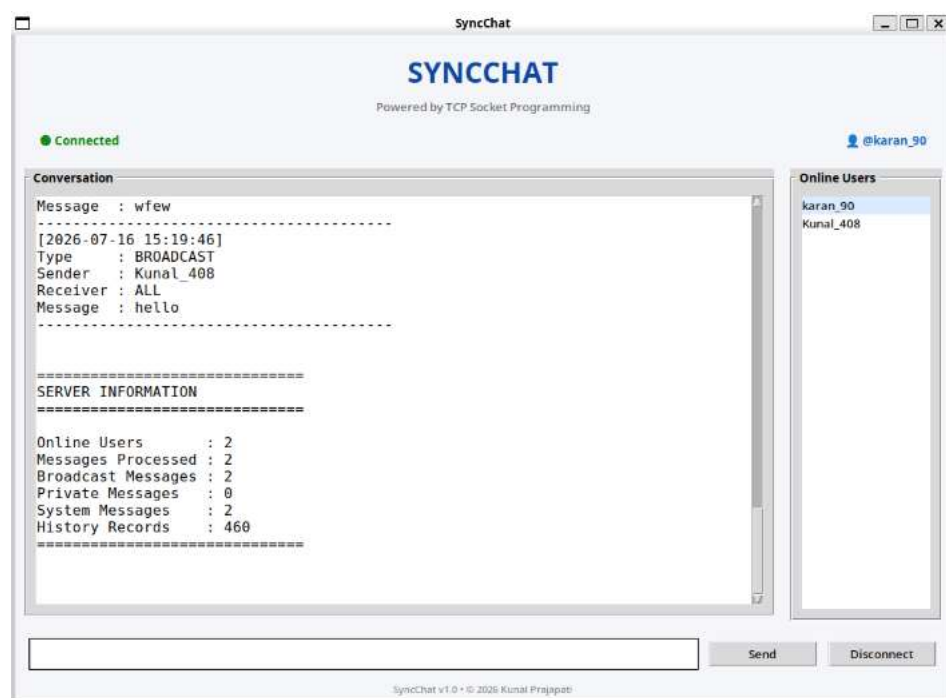
History Command

The **/history** command displayed recently stored chat messages from **chat_history.csv**.

Server Information Command

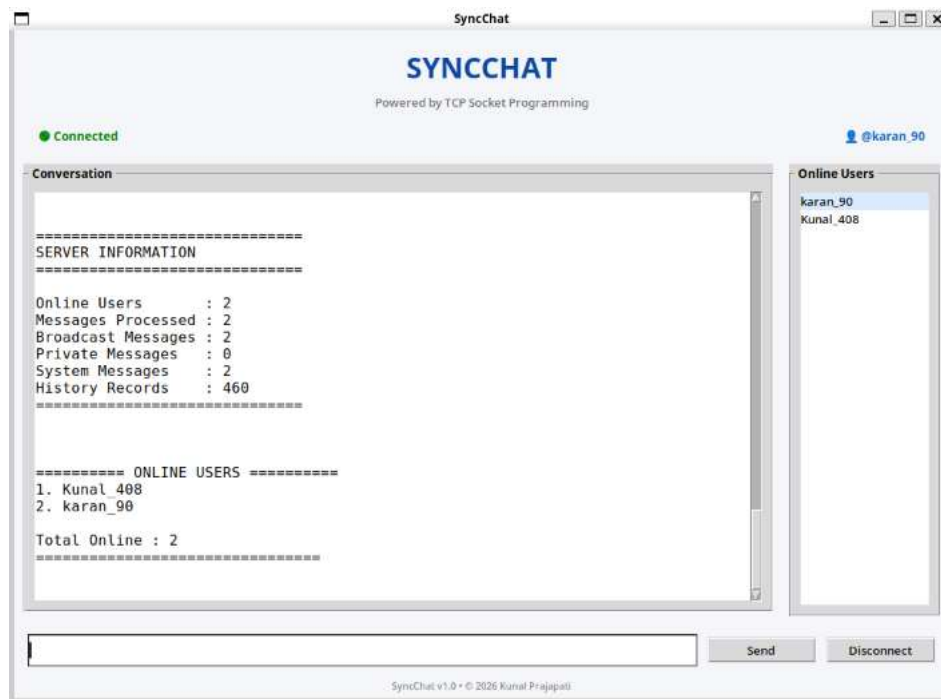


The **/info** command displayed server statistics including connected users, processed messages, broadcast count, private messages, and chat history records.



Online Users Command

The **/list** command successfully displayed all active users currently connected to the server.



11.7 Message Length Validation Testing

Oversized messages exceeding the predefined maximum limit were entered into the message input box.

The application correctly rejected such messages and displayed an informative warning dialog.

No oversized packets were transmitted to the server.

Observation

- Message limit enforced.
- No server crash.
- User informed appropriately.



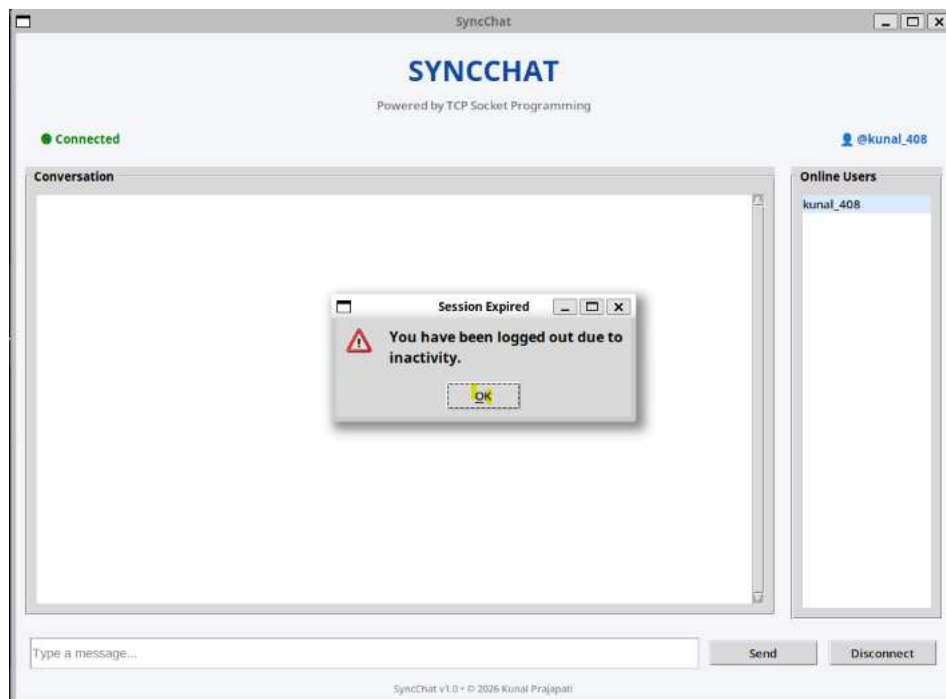
11.8 Session Timeout Testing

The inactivity monitoring mechanism was tested by leaving the application idle beyond the configured timeout duration.

After the timeout expired, the client was automatically disconnected from the server, and the event was recorded inside **security_log.txt**.

Observation

- Automatic logout successful.
- Security log updated.
- Session removed from active users.

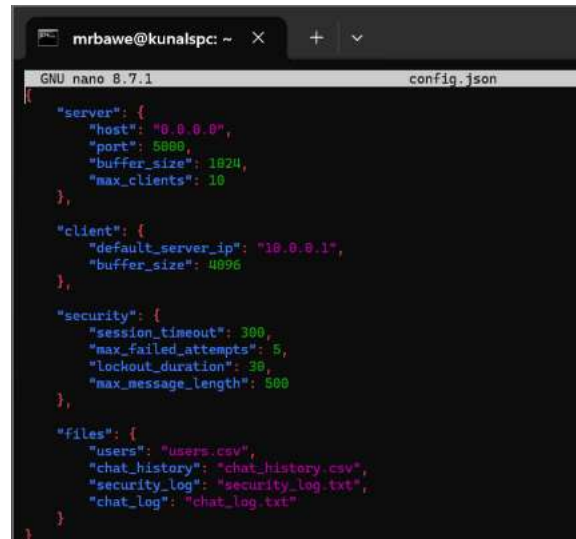


11.9 Configuration Management Testing

The centralized configuration mechanism was tested by modifying the network parameters stored in the configuration file before launching the application. The client and server successfully loaded the updated configuration without requiring any modifications to the application source code. This verified that deployment settings could be managed independently of the program logic, improving maintainability and deployment flexibility.

Observation

- Configuration loaded successfully.
- Server IP updated correctly.
- Port configuration applied successfully.
- No source code modification required.



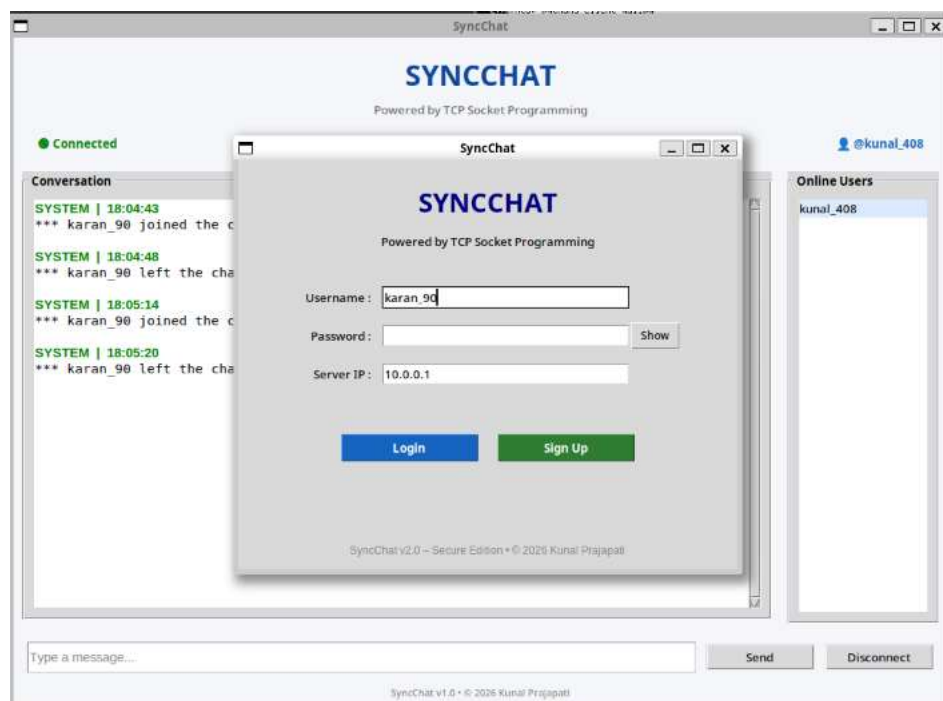
```
GNU nano 8.7.1 config.json
{
  "server": {
    "host": "0.0.0.0",
    "port": 5000,
    "buffer_size": 1024,
    "max_clients": 10
  },
  "client": {
    "default_server_ip": "10.0.0.1",
    "buffer_size": 4096
  },
  "security": {
    "session_timeout": 300,
    "max_failed_attempts": 5,
    "lockout_duration": 30,
    "max_message_length": 500
  },
  "files": {
    "users": "users.csv",
    "chat_history": "chat_history.csv",
    "security_log": "security_log.txt",
    "chat_log": "chat_log.txt"
  }
}
```

11.10 Connection Reliability Testing

The application was tested by intentionally disconnecting active clients during communication to evaluate connection reliability. The server successfully detected unexpected disconnections, removed inactive sessions, released associated resources, and continued serving the remaining connected users without interruption. No server crashes or communication failures were observed during testing.

Observation

- Graceful client disconnection.



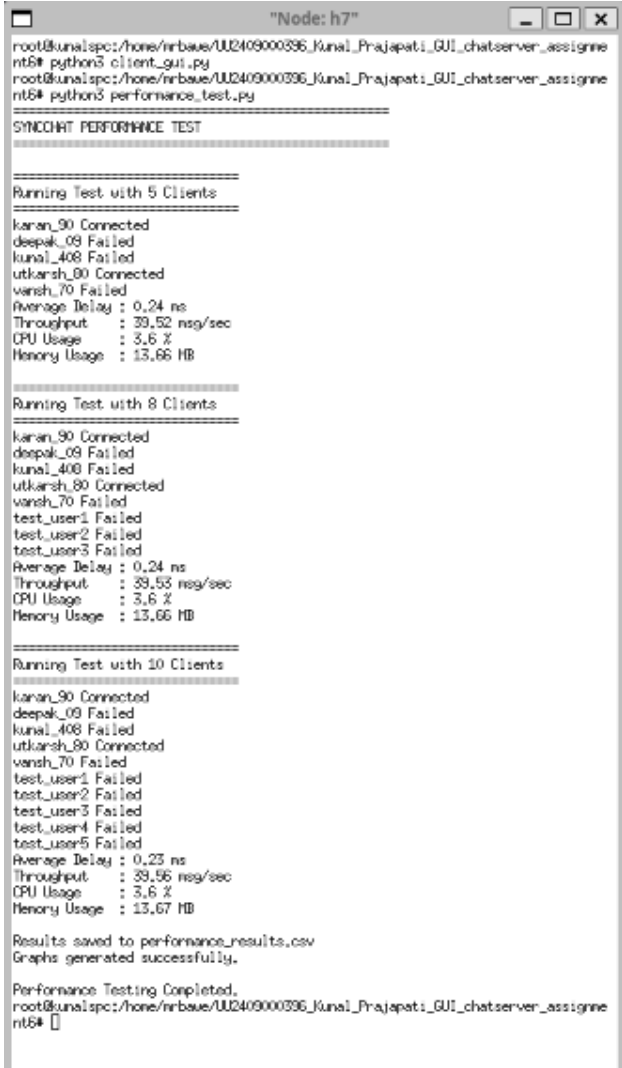
- Remaining users continued communication.
- Server remained operational.
- Active user list updated automatically.

11.11 Scalability Testing

Scalability testing was conducted by gradually increasing the number of concurrent clients connected to the server. Benchmark tests were performed with multiple client loads to evaluate the server's ability to handle simultaneous communication while maintaining reliable message delivery. Throughout the evaluation, the application remained stable, demonstrating its capability to support increasing numbers of active users.

Observation

- Multiple concurrent clients connected successfully.
- No communication failures observed.
- Stable server performance maintained.
- Message delivery remained reliable.



```

root@kunalisp:~/h7baue/U2409000396_Kunal_Prajapati_GUI_chatserver_assignme
nt6# python3 client_gui.py
root@kunalisp:~/h7baue/U2409000396_Kunal_Prajapati_GUI_chatserver_assignme
nt6# python3 performance_test.py

=====
SYNCHRONOUS PERFORMANCE TEST
=====

Running Test with 5 Clients
=====
karan_90 Connected
deepak_09 Failed
kunal_408 Failed
utkarsh_80 Connected
varsh_70 Failed
Average Delay : 0.24 ns
Throughput : 39.52 msg/sec
CPU Usage : 3.6 %
Memory Usage : 13.66 MB

Running Test with 8 Clients
=====
karan_90 Connected
deepak_09 Failed
kunal_408 Failed
utkarsh_80 Connected
varsh_70 Failed
test_user1 Failed
test_user2 Failed
test_user3 Failed
Average Delay : 0.24 ns
Throughput : 39.55 msg/sec
CPU Usage : 3.6 %
Memory Usage : 13.66 MB

Running Test with 10 Clients
=====
karan_90 Connected
deepak_09 Failed
kunal_408 Failed
utkarsh_80 Connected
varsh_70 Failed
test_user1 Failed
test_user2 Failed
test_user3 Failed
test_user4 Failed
test_user5 Failed
Average Delay : 0.23 ns
Throughput : 39.56 msg/sec
CPU Usage : 3.6 %
Memory Usage : 13.67 MB

Results saved to performance_results.csv
Graphs generated successfully.

Performance Testing Completed.
root@kunalisp:~/h7baue/U2409000396_Kunal_Prajapati_GUI_chatserver_assignme
nt6#

```

12. WIRESHARK PACKET ANALYSIS

Network communication between the client and server was analyzed using **Wireshark** to verify the correctness of TCP-based communication throughout the execution of the Secure SyncChat application. Packet analysis was performed during connection establishment, authentication, message transmission, user disconnection, and session termination to ensure reliable and secure communication between multiple clients and the centralized server.

In addition to verifying functional communication, packet captures were analyzed during scalability and performance testing to confirm that the application maintained reliable TCP communication while handling multiple concurrent client connections.

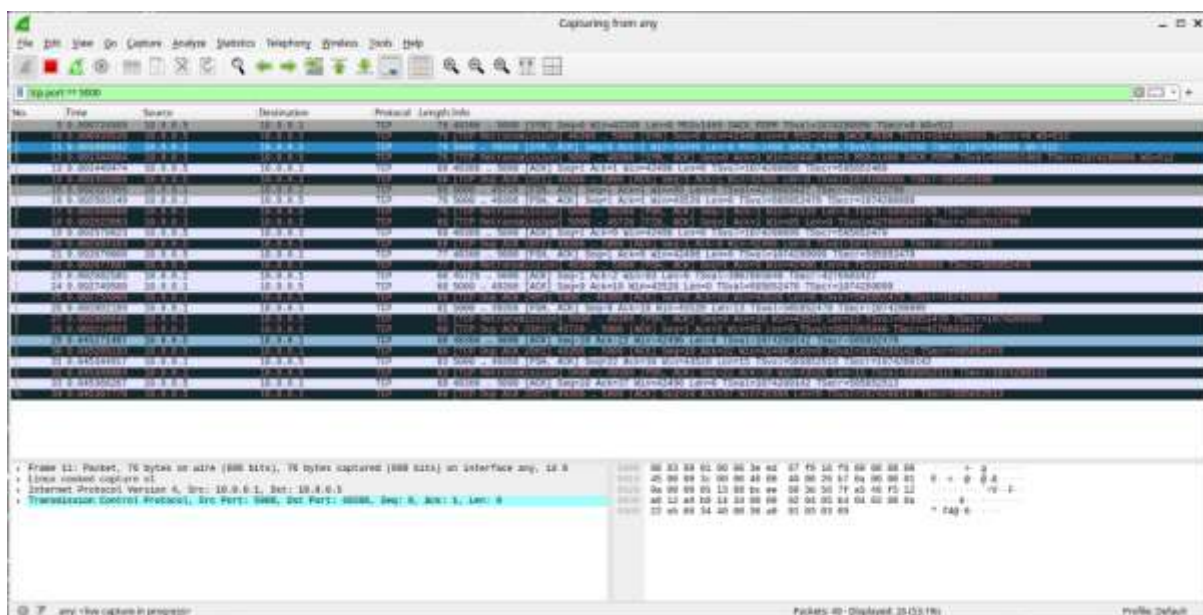
12.1 TCP Three-Way Handshake

Before communication begins, TCP establishes a reliable connection using the three-way handshake.

The captured packets clearly show:

- SYN
- SYN-ACK
- ACK

This confirms successful connection establishment between the client and the server.

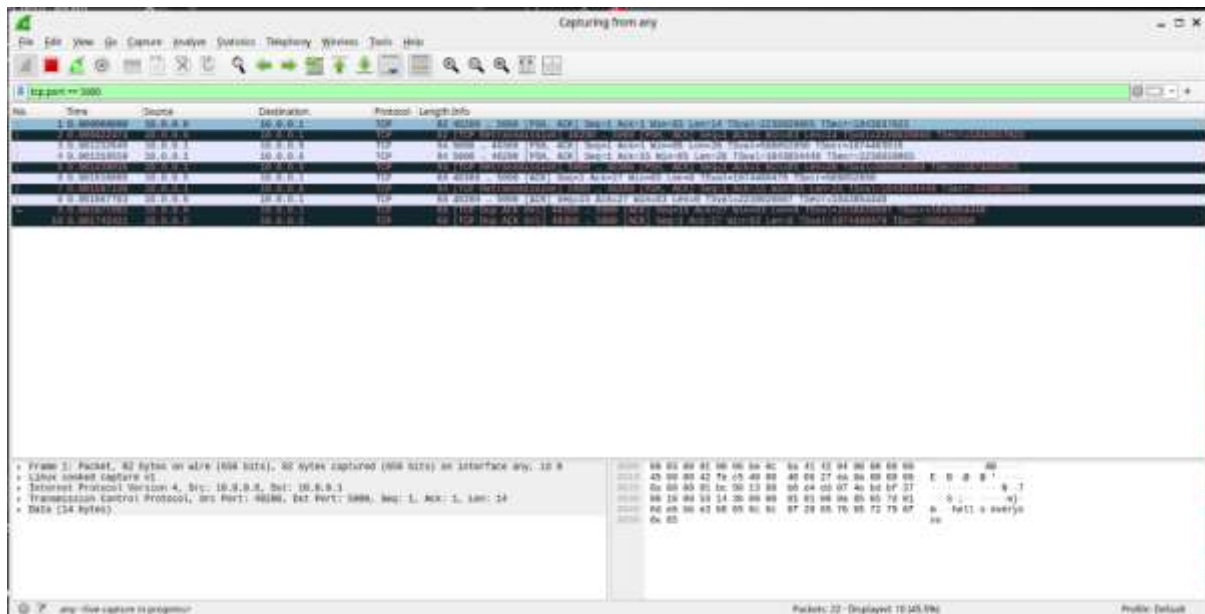


12.2 Broadcast Message Packet

A normal broadcast message was transmitted from one client.

Wireshark successfully captured the TCP payload containing the message.

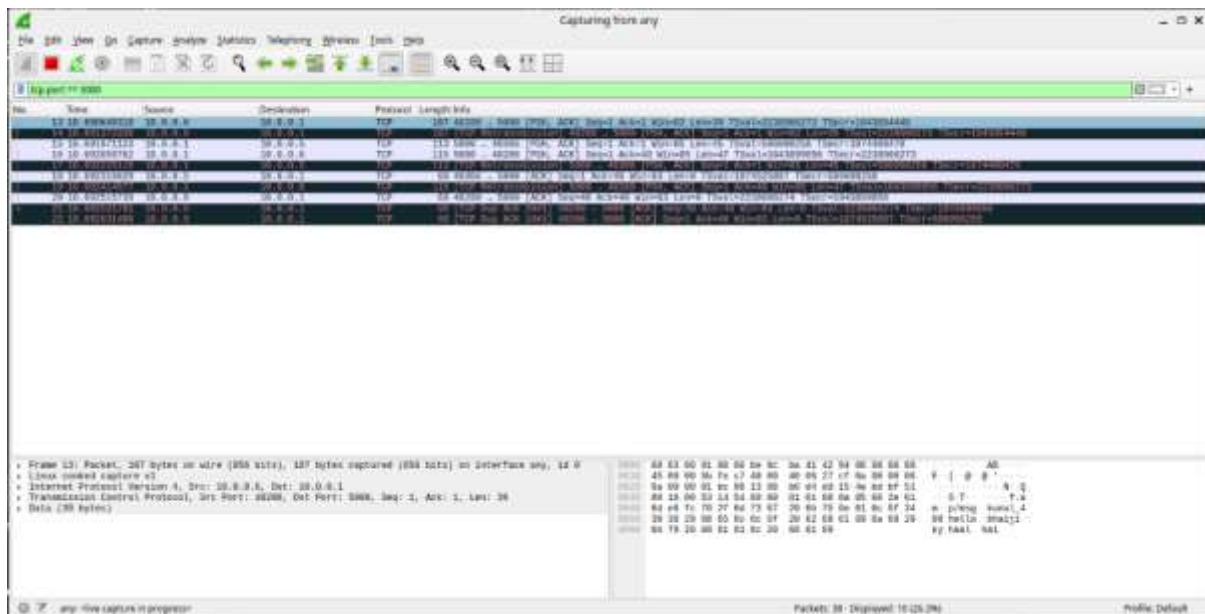
This verifies successful data transmission from the client to the server.



12.3 Private Message Packet

Private communication was analyzed using the /msg command.

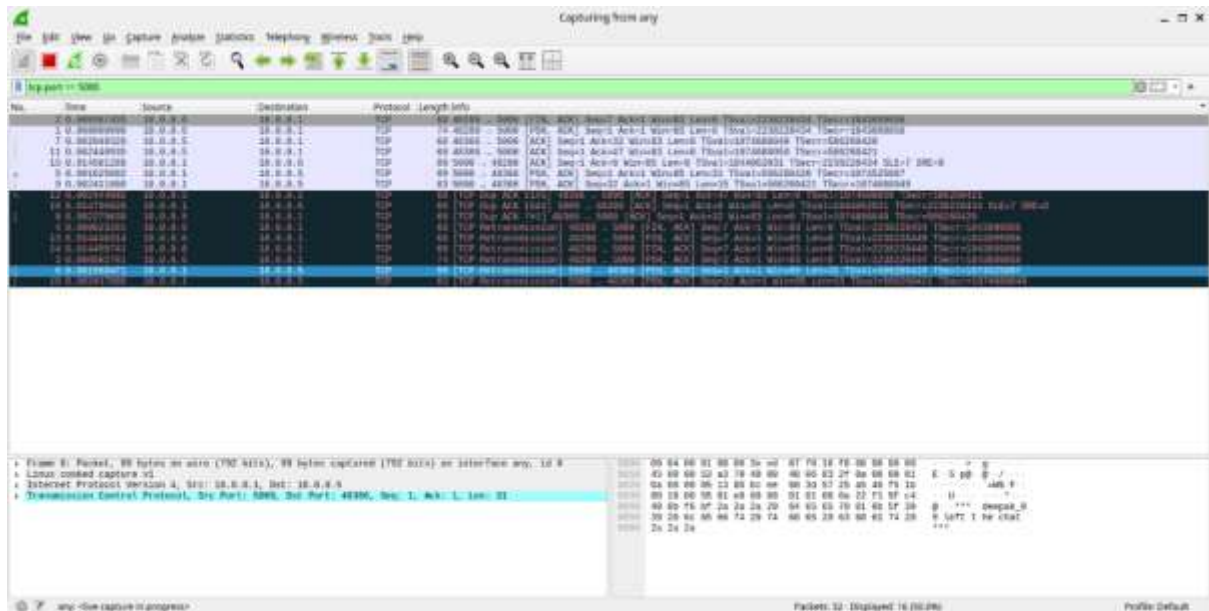
The packet capture confirmed successful delivery of the private message payload.



12.4 User Disconnection Packet

When a client disconnected, the server generated a notification packet informing remaining users.

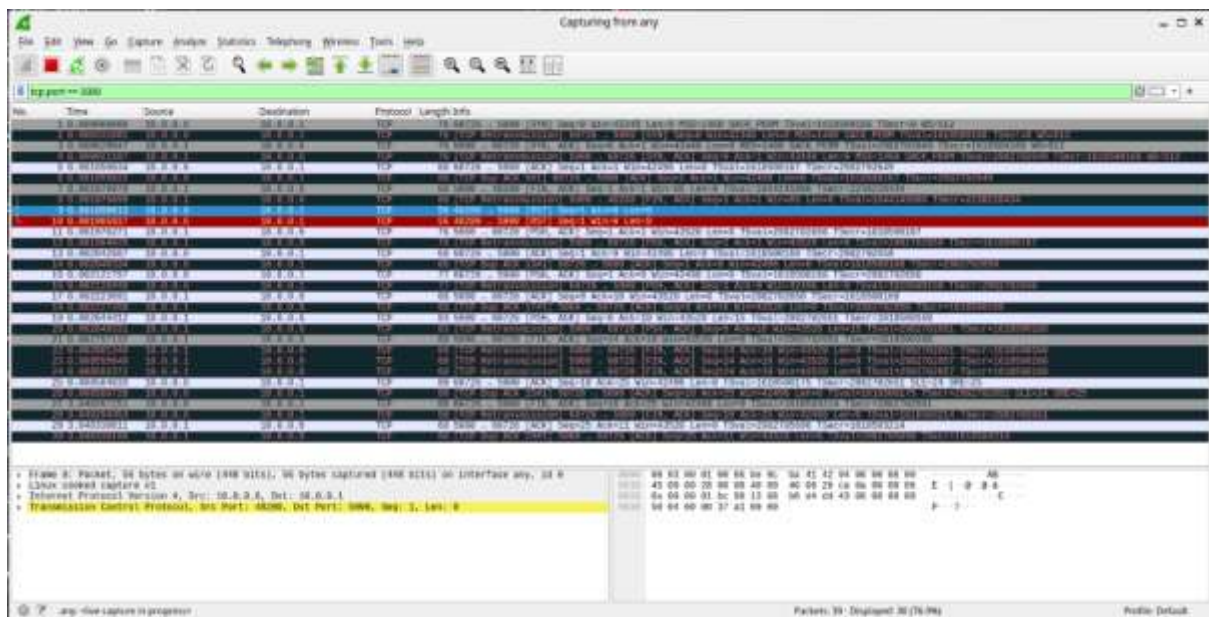
Wireshark successfully captured this communication.



12.5 TCP Connection Termination

When the client exited the application, TCP gracefully terminated the session.

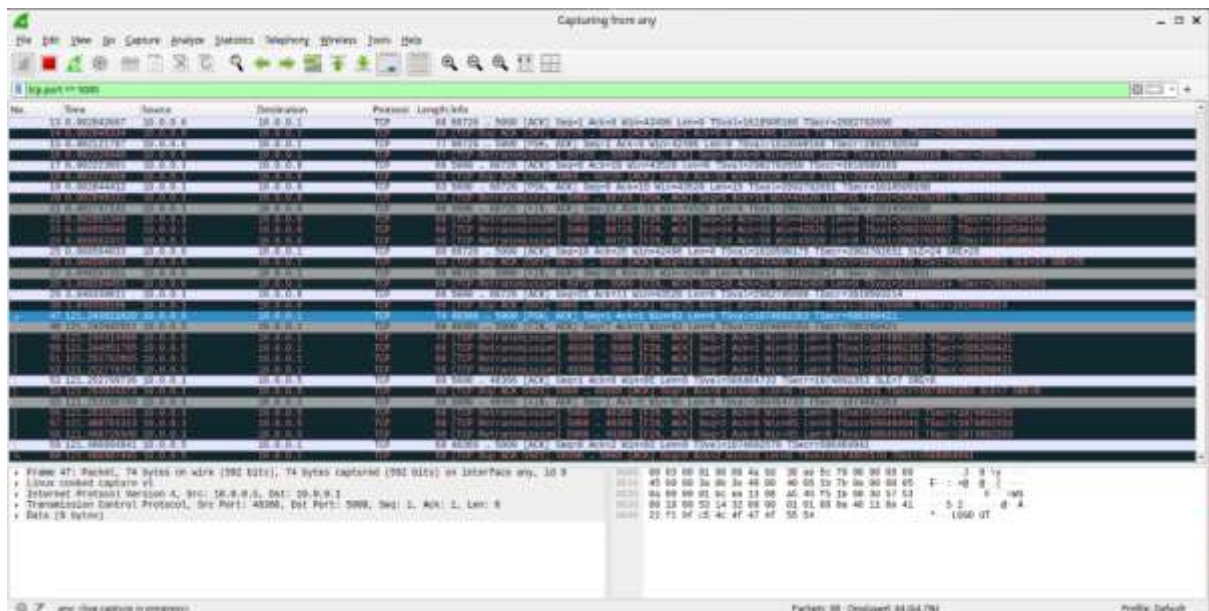
The FIN and ACK packets verified correct TCP connection termination.



12.6 Logout Packet

The client transmitted the **LOGOUT** command before disconnecting from the server.

This packet confirmed proper implementation of secure logout functionality.



13. LOG FILE ANALYSIS

To improve accountability, maintain persistent records, and support performance evaluation, the Secure SyncChat application automatically generates multiple log and configuration files during execution. These files assist in user authentication, communication history, security auditing, application configuration, and performance benchmarking. Together, they provide valuable information for monitoring application behavior, troubleshooting, and evaluating the reliability and scalability of the system.

13.1 User Database (users.csv)

The **users.csv** file functions as the credential database of the application. It stores usernames together with their corresponding SHA-256 hashed passwords.

Unlike traditional systems that store passwords in plain text, the implemented approach stores only cryptographic hash values, ensuring that original passwords cannot be retrieved directly from the database.

Information Stored

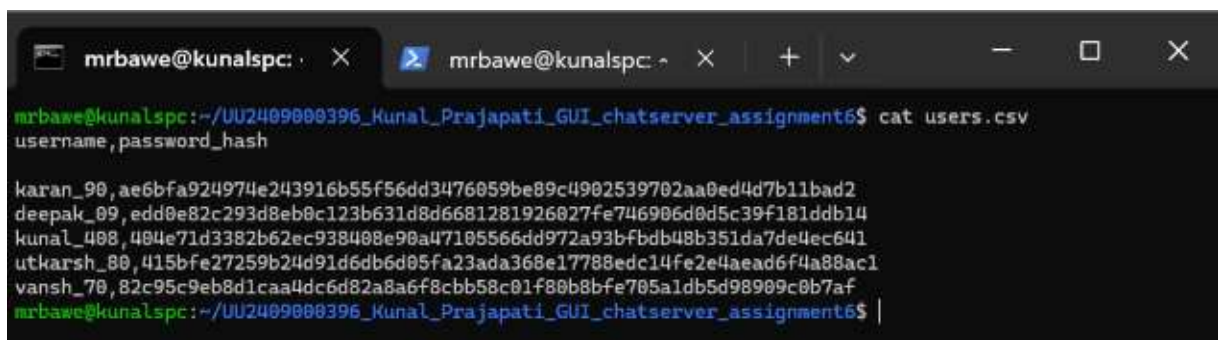
- Username
- SHA-256 Password Hash

Advantages

- Prevents plaintext password exposure.
- Improves credential security.
- Simplifies authentication.
- Lightweight database implementation.

Observation

All registered users were successfully stored inside the database using SHA-256 hashing. No plaintext passwords were present in the file.



```
mrbaawe@kunalspc: ~ - ssh - mrbaawe@kunalspc: ~  
mrbaawe@kunalspc:~/UU2409080396_Kunal_Prajapati_GUI_chatserver_assignment6$ cat users.csv  
username,password_hash  
  
karan_90,ae6bfa924974e243916b55f56dd3476059be89c4902539702aa0ed4d7b11bad2  
deepak_09,edd0e82c293d8eb0c123b631d8d66812819260277fe746906d0d5c39f181ddb14  
kunal_408,404e71d3382b62ec938408e90a47105566dd972a93bfbbd48b351da7de4ec641  
utkarsh_80,415bfe27259b24d91d6db6d05fa23ada368e17788edc14fe2e4aeadd6f4a88ac1  
vansh_70,82c95c9eb8d1caa4dc6d82a8a6f8cbb58c01f80b8bfe705a1db5d98909c0b7af  
mrbaawe@kunalspc:~/UU2409080396_Kunal_Prajapati_GUI_chatserver_assignment6$ |
```

13.2 Chat History (chat_history.csv)

The application automatically records every successfully transmitted message inside **chat_history.csv**.

Each record includes detailed information regarding communication between users.

Information Stored

- Timestamp
- Message Type
- Sender
- Receiver
- Message Content

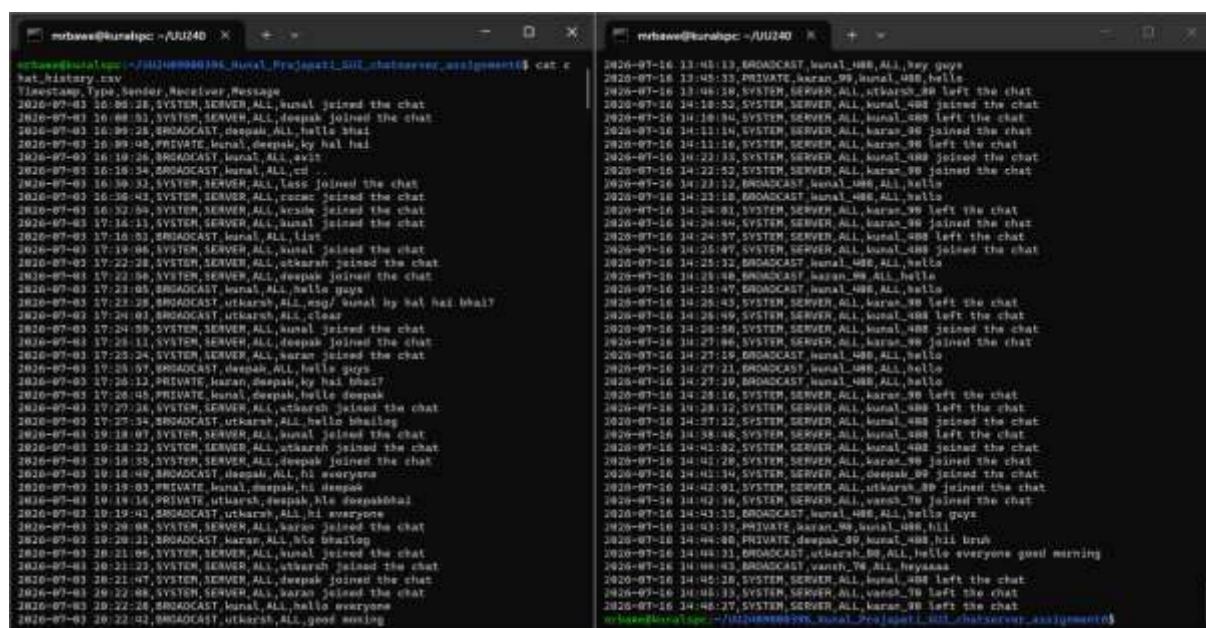
Maintaining persistent chat history enables users and administrators to review previous communications even after restarting the application.

Advantages

- Permanent communication record.
- Easy auditing.
- Message tracking.
- Historical analysis.

Observation

All broadcast messages, private messages, login notifications, and logout notifications were correctly recorded.



```
utkarsh@kunalpc: ~/JUJ240
utkarsh@kunalpc:~/JUJ240$ cat chat_history.csv
Timestamp,Type,Sender,Receiver,Message
2020-07-03 10:00:25,SYSTEM,SERVER,ALL,kunal_488 joined the chat
2020-07-03 10:00:31,SYSTEM,SERVER,ALL,deepak joined the chat
2020-07-03 10:00:35,BROADCAST,deepak,ALL,hello hai
2020-07-03 10:00:40,PRIVATE,kunal,deepak,hi hai hai
2020-07-03 10:10:26,BROADCAST,kunal,ALL,exit
2020-07-03 10:10:34,BROADCAST,kunal,ALL,hi
2020-07-03 10:10:32,SYSTEM,SERVER,ALL,less joined the chat
2020-07-03 10:10:41,SYSTEM,SERVER,ALL,rcac joined the chat
2020-07-03 10:10:44,SYSTEM,SERVER,ALL,kacade joined the chat
2020-07-03 17:16:11,SYSTEM,SERVER,ALL,kunal joined the chat
2020-07-03 17:10:03,BROADCAST,kunal,ALL,less
2020-07-03 17:24:05,SYSTEM,SERVER,ALL,kunal joined the chat
2020-07-03 17:22:28,SYSTEM,SERVER,ALL,utkarsh joined the chat
2020-07-03 17:22:36,SYSTEM,SERVER,ALL,deepak joined the chat
2020-07-03 17:23:05,BROADCAST,kunal,ALL,hello guys
2020-07-03 17:23:29,BROADCAST,utkarsh,ALL,msg/ kunal hi hai hai bhai?
2020-07-03 17:24:04,BROADCAST,utkarsh,ALL,less
2020-07-03 17:24:05,SYSTEM,SERVER,ALL,kunal joined the chat
2020-07-03 17:25:11,SYSTEM,SERVER,ALL,deepak joined the chat
2020-07-03 17:25:24,SYSTEM,SERVER,ALL,karan joined the chat
2020-07-03 17:25:57,BROADCAST,deepak,ALL,hello guys
2020-07-03 17:26:12,PRIVATE,karan,deepak,hi hai bhai?
2020-07-03 17:26:15,PRIVATE,kunal,deepak,hello deepak
2020-07-03 17:27:26,SYSTEM,SERVER,ALL,utkarsh joined the chat
2020-07-03 17:27:34,BROADCAST,utkarsh,ALL,hello bhaiing
2020-07-03 19:18:07,SYSTEM,SERVER,ALL,kunal joined the chat
2020-07-03 19:18:22,SYSTEM,SERVER,ALL,utkarsh joined the chat
2020-07-03 19:18:35,SYSTEM,SERVER,ALL,deepak joined the chat
2020-07-03 19:18:49,BROADCAST,deepak,ALL,hi everyone
2020-07-03 19:19:03,PRIVATE,kunal,deepak,hi deepak
2020-07-03 19:19:16,PRIVATE,utkarsh,deepak,hi deepakbhai
2020-07-03 19:19:41,BROADCAST,utkarsh,ALL,hi everyone
2020-07-03 19:20:08,SYSTEM,SERVER,ALL,karan joined the chat
2020-07-03 19:20:11,BROADCAST,karan,ALL,hiu bhaiing
2020-07-03 20:21:06,SYSTEM,SERVER,ALL,kunal joined the chat
2020-07-03 20:21:23,SYSTEM,SERVER,ALL,utkarsh joined the chat
2020-07-03 20:21:47,SYSTEM,SERVER,ALL,deepak joined the chat
2020-07-03 20:22:08,SYSTEM,SERVER,ALL,karan joined the chat
2020-07-03 20:22:28,BROADCAST,kunal,ALL,hello everyone
2020-07-03 20:22:42,BROADCAST,utkarsh,ALL,good morning

2020-07-16 13:45:13,BROADCAST,kunal_488,ALL,hey guys
2020-07-16 13:45:33,PRIVATE,karan_99,kunal_488,hello
2020-07-16 13:46:10,SYSTEM,SERVER,ALL,utkarsh_80 left the chat
2020-07-16 14:10:22,SYSTEM,SERVER,ALL,kunal_488 joined the chat
2020-07-16 14:10:34,SYSTEM,SERVER,ALL,kunal_488 left the chat
2020-07-16 14:11:14,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:11:16,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:22:33,SYSTEM,SERVER,ALL,kunal_488 joined the chat
2020-07-16 14:22:52,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:23:12,BROADCAST,kunal_488,ALL,hello
2020-07-16 14:23:18,BROADCAST,kunal_488,ALL,hello
2020-07-16 14:24:01,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:24:44,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:24:57,SYSTEM,SERVER,ALL,kunal_488 left the chat
2020-07-16 14:25:07,SYSTEM,SERVER,ALL,kunal_488 joined the chat
2020-07-16 14:25:32,BROADCAST,kunal_488,ALL,hello
2020-07-16 14:25:46,BROADCAST,karan_99,ALL,hello
2020-07-16 14:25:47,BROADCAST,kunal_488,ALL,hello
2020-07-16 14:26:43,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:26:49,SYSTEM,SERVER,ALL,kunal_488 left the chat
2020-07-16 14:26:58,SYSTEM,SERVER,ALL,kunal_488 joined the chat
2020-07-16 14:27:06,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:27:16,BROADCAST,kunal_488,ALL,hello
2020-07-16 14:27:21,BROADCAST,kunal_488,ALL,hello
2020-07-16 14:27:39,BROADCAST,kunal_488,ALL,hello
2020-07-16 14:28:10,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:28:32,SYSTEM,SERVER,ALL,kunal_488 left the chat
2020-07-16 14:27:06,SYSTEM,SERVER,ALL,kunal_488 joined the chat
2020-07-16 14:27:22,SYSTEM,SERVER,ALL,kunal_488 left the chat
2020-07-16 14:28:48,SYSTEM,SERVER,ALL,kunal_488 left the chat
2020-07-16 14:41:02,SYSTEM,SERVER,ALL,kunal_488 joined the chat
2020-07-16 14:41:20,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:42:14,SYSTEM,SERVER,ALL,deepak_89 joined the chat
2020-07-16 14:42:01,SYSTEM,SERVER,ALL,utkarsh_80 joined the chat
2020-07-16 14:42:36,SYSTEM,SERVER,ALL,vansh_70 joined the chat
2020-07-16 14:43:10,BROADCAST,kunal_488,ALL,hello guys
2020-07-16 14:43:33,PRIVATE,karan_99,kunal_488,hi
2020-07-16 14:44:30,PRIVATE,deepak_89,kunal_488,hi hihi
2020-07-16 14:44:31,BROADCAST,utkarsh_80,ALL,hello everyone good morning
2020-07-16 14:44:43,BROADCAST,vansh_70,ALL,heyaaa
2020-07-16 14:45:16,SYSTEM,SERVER,ALL,kunal_488 left the chat
2020-07-16 14:45:33,SYSTEM,SERVER,ALL,vansh_70 left the chat
2020-07-16 14:45:37,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:45:37,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:45:37,SYSTEM,SERVER,ALL,karan_99 left the chat
```

13.3 Security Log (security_log.txt)

A dedicated security log file was implemented to separately record security-related events occurring during application execution.

Unlike chat history, this log focuses exclusively on authentication and security operations.

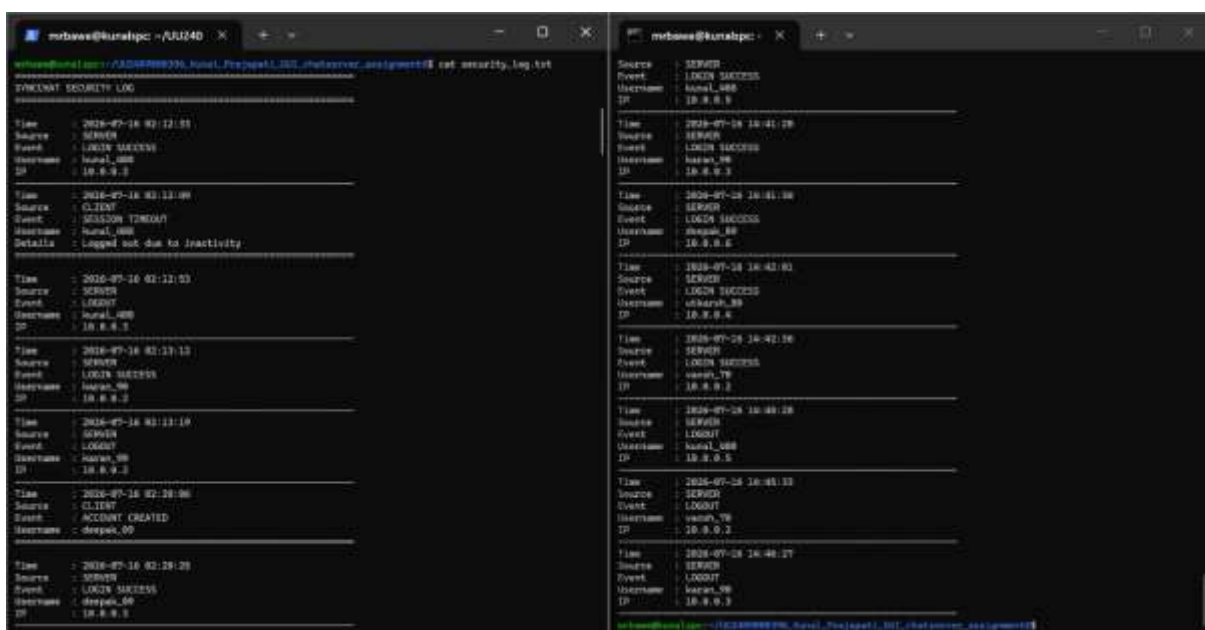
Recorded Events

- Account Creation
- Login Success
- Duplicate Login Attempt
- Logout
- Session Timeout
- Failed Login Attempt

Maintaining a separate security log improves traceability and allows administrators to investigate suspicious user activity efficiently.

Advantages

- Security auditing.
- User activity tracking.
- Login monitoring.
- Session monitoring.
- Attack detection support.



Observation

Every important security event was successfully recorded with the corresponding timestamp and user information.

13.4 Server Runtime Statistics

During execution, the server continuously displayed runtime statistics that helped monitor application performance.

The displayed statistics included:

- Number of Connected Users
- Messages Processed
- Broadcast Messages
- Private Messages
- System Messages
- CPU Utilization (during benchmark testing)
- Memory Utilization (during benchmark testing)

```
"Node: h1"
root@kunalpc:/home/mbawa/IIU240S000396_Kunal_Prajapati_GUI_chatserver_assignme
nt04# python3 server.py
Advanced Chat Server started on port 5000

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 1
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 1
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 2
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 2
=====

Handle Error : [Errno 104] Connection reset by peer

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 1
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 3
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 2
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 4
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 3
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 5
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 4
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 6
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 0
Broadcast Messages : 0
```

```
"Node: h1"

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 2
Broadcast Messages : 2
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 3
Broadcast Messages : 3
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 4
Broadcast Messages : 4
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 5
Broadcast Messages : 5
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 6
Broadcast Messages : 6
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 7
Broadcast Messages : 7
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 8
Broadcast Messages : 8
Private Messages  : 1
System Messages   : 7
=====
```

These statistics were automatically updated whenever users connected, disconnected, or exchanged messages.

Observation

The displayed statistics accurately reflected the current state of the application throughout the testing process. During benchmark execution, resource utilization metrics were also monitored to evaluate system performance under varying client workloads.

13.5 Configuration File (config.json)

To simplify deployment and improve maintainability, the application uses a centralized configuration file named config.json. Instead of hardcoding network parameters into the source code, important configuration settings are stored externally, allowing the application to be deployed in different environments without modifying the program.

Information Stored

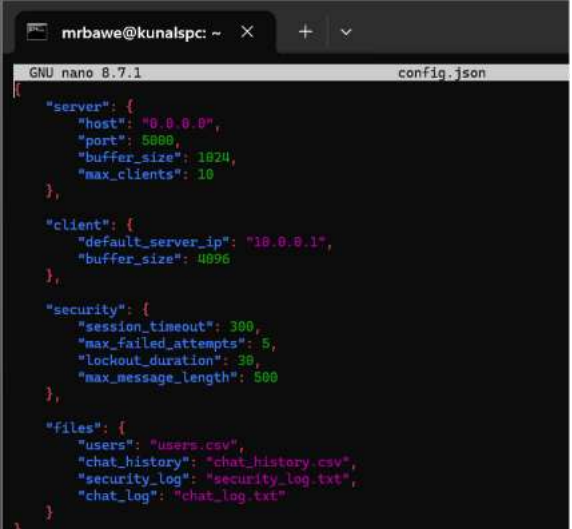
- Server IP Address
- Server Port Number
- Buffer Size
- Maximum Client Connections
- Application Configuration Parameters

Advantages

- Centralized configuration management
- Easy deployment across different systems
- Reduced source code modification
- Improved maintainability
- Flexible network configuration

Observation

The application successfully loaded all configuration parameters during startup, and changes made to the configuration file were reflected without requiring modifications to the application source code.



```
GNU nano 8.7.1 config.json
{
  "server": {
    "host": "0.0.0.0",
    "port": 5000,
    "buffer_size": 1024,
    "max_clients": 10
  },
  "client": {
    "default_server_ip": "10.0.0.1",
    "buffer_size": 4096
  },
  "security": {
    "session_timeout": 300,
    "max_failed_attempts": 5,
    "lockout_duration": 30,
    "max_message_length": 500
  },
  "files": {
    "users": "users.csv",
    "chat_history": "chat_history.csv",
    "security_log": "security_log.txt",
    "chat_log": "chat_log.txt"
  }
}
```

13.6 Performance Results File (performance_results.csv)

A dedicated performance results file was generated during benchmark execution to record important performance metrics under different client workloads. The recorded information was later used to generate graphical representations for analyzing application efficiency, scalability, and resource utilization.

Information Stored

- Number of Connected Clients
- Average Message Delay
- Message Throughput
- CPU Utilization
- Memory Utilization

Advantages

- Performance benchmarking
- Scalability evaluation
- Resource utilization analysis
- Graph generation
- Experimental result documentation

Observation

Performance results were successfully recorded for each benchmark execution and used to generate graphs demonstrating the application's scalability and optimization.

A	B	C	D	E	F	G
Clients	Average D Throughput	CPU Usage	Memory Usage (MB)			
5	0.24	39.52	3.6	13.66		
8	0.24	39.53	3.6	13.66		
10	0.23	39.56	3.6	13.67		

13.7 Benchmark Graph Analysis

The recorded benchmark data was analyzed using graphical representations to evaluate application performance under different client workloads. Individual graphs illustrating message delay, throughput, CPU utilization, and memory consumption were generated from the benchmark results. These visualizations helped verify that the application maintained

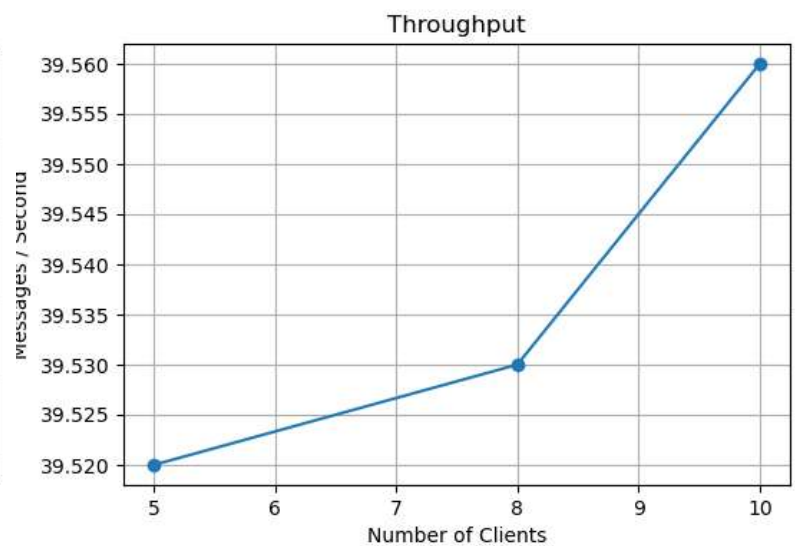
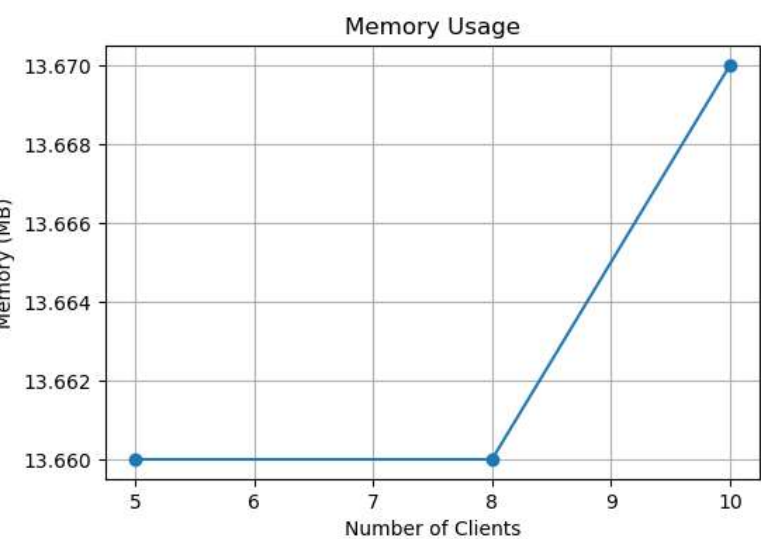
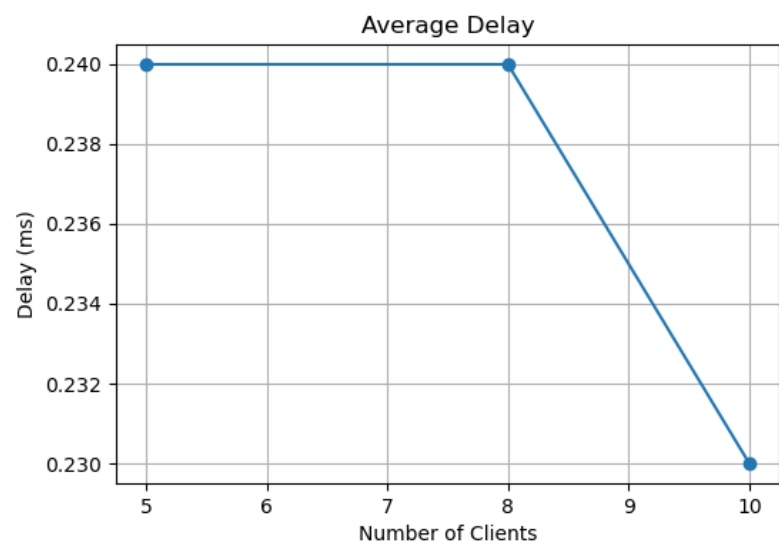
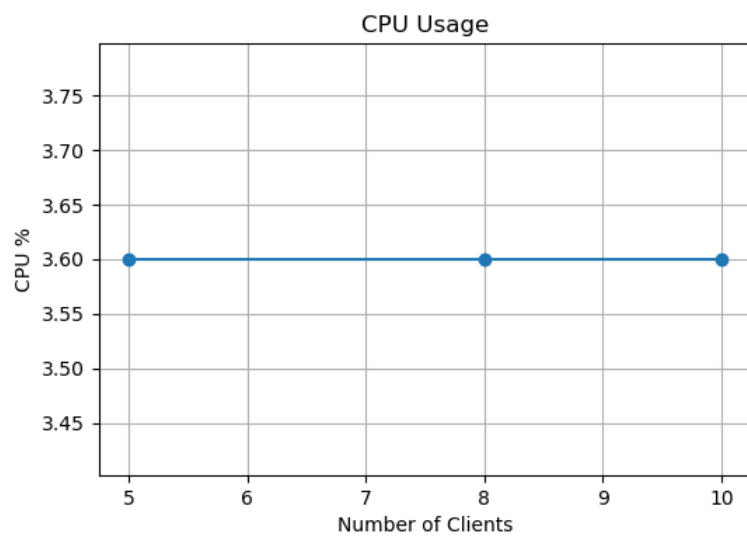
stable performance and efficient resource utilization while supporting multiple concurrent clients.

Advantages

- Simplified performance interpretation
- Scalability verification
- Resource monitoring
- Comparative workload analysis

Observation

The generated graphs confirmed that the application maintained reliable communication with acceptable resource utilization as the number of concurrent clients increased.



14. CHALLENGES FACED

During the development and enhancement of the Secure SyncChat application, several practical challenges were encountered while implementing networking, security, reliability, scalability, and performance optimization features. Each challenge required careful analysis, debugging, and iterative improvements before arriving at the final implementation. Overcoming these challenges significantly improved the stability, maintainability, and overall performance of the application.

Challenge 1: GUI Integration with Socket Communication

Initially, integrating Tkinter's event-driven interface with continuous socket communication caused the GUI to become unresponsive. This issue was resolved by implementing separate receiver threads, allowing the GUI and networking components to execute simultaneously.

Challenge 2: Duplicate Login Prevention

Maintaining a synchronized list of active users while preventing duplicate logins required careful server-side session management. The server was modified to verify active usernames before allowing new connections.

Challenge 3: Session Timeout

Implementing automatic inactivity logout while ensuring that active users remained connected proved challenging. A background timer was developed to monitor user activity and automatically disconnect inactive sessions.

Challenge 4: Online User Synchronization

Synchronizing the online user list across multiple clients required the server to continuously broadcast updated user lists whenever users connected or disconnected.

Challenge 5: Security Logging

A dedicated logging mechanism was designed to record authentication events separately from normal chat messages, improving traceability and simplifying security auditing.

Challenge 6: Wireshark Packet Analysis

Capturing only application-specific packets initially resulted in unnecessary network traffic appearing during analysis. Applying TCP port filtering enabled clear visualization of login packets, broadcast messages, private communication, and TCP connection termination.

Challenge 7: Centralized Configuration Management

Introducing a centralized configuration file required redesigning the application's initialization process so that both the client and server could dynamically load network parameters during startup. Ensuring that configuration changes were correctly applied without affecting existing functionality required careful validation and testing across different execution environments.

Challenge 8: Performance Benchmark Development

Designing a benchmarking module capable of measuring message delay, throughput, CPU utilization, and memory consumption while multiple clients communicated simultaneously proved challenging. Accurate metric collection required synchronizing client activity with performance monitoring while minimizing the measurement overhead introduced by the benchmarking process itself.

Challenge 9: Scalability Testing

Evaluating the application's behavior under increasing client workloads required configuring multiple concurrent client instances within the Mininet environment. Maintaining reliable communication, preserving message ordering, and preventing server instability during high client loads required continuous optimization of the server's multi-threaded communication model.

Challenge 10: Performance Analysis and Visualization

After collecting benchmark data, the recorded performance metrics had to be organized into a structured format suitable for graphical analysis. Generating meaningful visualizations for message delay, throughput, CPU utilization, and memory consumption while ensuring consistency between recorded values and generated graphs required careful data processing and verification.

15. LEARNING OUTCOMES

The successful completion of this project significantly enhanced practical knowledge in the fields of computer networking, cybersecurity, software engineering, performance optimization, and system scalability. The development and evaluation of the Secure SyncChat application provided valuable hands-on experience in designing, implementing, testing, and optimizing a secure multi-client communication system.

The major learning outcomes include:

- Understanding of TCP socket programming.
- Development of multi-threaded client-server applications.
- GUI development using the Tkinter framework.
- Implementation of secure user authentication mechanisms.
- Password hashing using the SHA-256 cryptographic algorithm.
- User session management and inactivity monitoring.
- Duplicate login prevention and secure access control.
- Secure event logging and activity auditing.
- Online user synchronization.
- Broadcast and private messaging implementation.
- Centralized configuration management using configuration files.
- Reliable connection handling and exception management.
- Performance benchmarking and system evaluation.
- Scalability testing using multiple concurrent clients.
- Performance analysis using throughput, message delay, CPU utilization, and memory consumption metrics.
- Generation and interpretation of performance graphs.
- Wireshark-based TCP packet analysis and protocol verification.
- Practical debugging of distributed client-server applications.
- Modular software design and implementation.
- Application testing and validation in a virtual network environment.
- Adoption of secure software engineering and performance optimization practices.

The project also strengthened analytical thinking, debugging skills, and problem-solving abilities by addressing real-world challenges related to secure communication, reliability, scalability, and system performance. The experience gained through implementing, testing, and optimizing the application provides a strong foundation for developing secure, efficient, and scalable network-based software systems in professional environments.

16. CONCLUSION

The Secure **SyncChat** application was successfully designed, developed, and evaluated as a secure GUI-based multi-client communication system using Python socket programming. The application integrates networking, cybersecurity, graphical user interface development, and software engineering principles to provide a reliable and user-friendly communication platform.

The developed system provides a graphical interface for user registration, secure authentication, and real-time communication while ensuring secure credential storage through SHA-256 password hashing. Security mechanisms such as duplicate login prevention, login attempt restriction, session timeout, secure event logging, chat history maintenance, and online user synchronization enhance the confidentiality, integrity, and reliability of the communication system.

In addition to the core communication and security features, the application was further enhanced through centralized configuration management, improved connection handling, exception management, and support for scalable multi-client communication. Benchmark testing demonstrated the system's ability to maintain stable performance under varying client workloads, while performance metrics including message delay, throughput, CPU utilization, and memory consumption were successfully collected and analyzed using graphical representations.

Comprehensive experimental testing verified the correct operation of all implemented functionalities, including user registration, secure login, broadcast messaging, private messaging, server commands, session management, configuration management, scalability testing, and automated performance benchmarking. Network packet analysis using Wireshark further confirmed reliable TCP connection establishment, secure message transmission, graceful connection termination, and stable communication throughout the testing process.

Overall, the project demonstrates the practical application of computer networking, Python programming, GUI development, cybersecurity, configuration management, performance optimization, and system scalability within a real-world client-server environment. The knowledge and experience gained throughout the development, testing, and optimization of the Secure SyncChat application provide a strong foundation for designing secure, reliable, and scalable network-based software systems and can be further extended with advanced features such as database integration, end-to-end encryption, cloud deployment, and mobile platform support.

17. REFERENCES

1. □ Python Software Foundation. *Python 3 Documentation*. Available at: <https://docs.python.org/>
 2. □ Python Software Foundation. *Tkinter — Python Interface to Tcl/Tk*. Available at: <https://docs.python.org/3/library/tkinter.html>
 3. □ Python Software Foundation. *Socket Programming Documentation*. Available at: <https://docs.python.org/3/library/socket.html>
 4. □ Python Software Foundation. *Threading Module Documentation*. Available at: <https://docs.python.org/3/library/threading.html>
 5. □ Python Software Foundation. *CSV File Processing Documentation*. Available at: <https://docs.python.org/3/library/csv.html>
 6. □ Python Software Foundation. *JSON Module Documentation*. Available at: <https://docs.python.org/3/library/json.html>
 7. □ Wireshark Foundation. *Wireshark User Guide*. Available at: <https://www.wireshark.org/docs/>
 8. □ Mininet Project. *Mininet Network Emulator Documentation*. Available at: <http://mininet.org/>
 9. □ National Institute of Standards and Technology (NIST). *Secure Hash Standard (SHA-256) (FIPS PUB 180-4)*. Available at: <https://csrc.nist.gov/publications/detail/fips/180/4/final>
 10. □ Python Software Foundation. *Time Module Documentation*. Available at: <https://docs.python.org/3/library/time.html>
 11. □ Python Software Foundation. *Logging Module Documentation*. Available at: <https://docs.python.org/3/library/logging.html>
 12. □ Python Software Foundation. *OS Module Documentation*. Available at: <https://docs.python.org/3/library/os.html>
 13. □ Python Software Foundation. *Resource Management and Performance Monitoring Documentation*. Available at: <https://docs.python.org/3/library/resource.html> (Linux/Unix environments)
 14. □ Matplotlib Development Team. *Matplotlib Documentation*. Available at: <https://matplotlib.org/stable/>
 15. □ ISEA Phase-III Internship Assignment Manual.
-